

ECOLE POLYTECHNIQUE DE YAOUNDE
Departement Genie Informatique — 5^{eme} Annee

FleetMan

Plateforme de Gestion de Flotte de Vehicules

Rapport d'Analyse et de Conception

Architecture Hexagonale Reactive — Backend Spring Boot

Theme : **Gestion de Flotte Multi-Tenant**

Stack : **Spring Boot 3.2 · WebFlux · PostgreSQL · React**

Promotion : **5GI — Depot 1 — Annee 2025–2026**

Encadrant : **[Nom de l'encadrant]**

Table des matières

Resume	iv
Introduction Generale	1
I Etat de l’Art et Analyse de l’Existant	3
1 Etat de l’Art des Solutions de Gestion de Flotte	4
1.1 Justification des criteres d’evaluation	4
1.2 Positionnement de FleetMan	5
2 Analyse des Besoins	6
2.1 Acteurs du systeme	6
2.2 Diagramme de cas d’utilisation	6
2.2.1 Analyse detaillee des cas d’utilisation	6
2.2.2 Matrice de tracabilite acteurs-fonctionnalites	10
2.3 Exigences non fonctionnelles	11
II Conception de l’Architecture	12
3 Choix Technologiques et Architecture Hexagonale	13
3.1 Justification des choix technologiques	13
3.2 Architecture hexagonale — Vue d’ensemble	13
3.3 Diagramme de packages	13
3.3.1 Justification de l’architecture en packages	14
3.3.2 Avantages de cette organisation	14
3.3.3 Exemple concret de flux inter-packages	16
3.4 Diagramme de composants	16
3.4.1 Composants principaux et responsabilites	16
3.4.2 Patterns d’integration avec services externes	19
4 Modele de Donnees et Classes Metier	20
4.1 Diagramme de classes — Couche domaine	20
4.1.1 Entites principales et leurs responsabilites	20
4.1.2 Patterns DDD appliques	24

4.2	Diagramme de classes — Ports et Adapters	24
4.2.1	Ports entrants (Inbound Ports / Use Cases)	24
4.2.2	Ports sortants (Outbound Ports)	25
4.2.3	Avantages du pattern Ports & Adapters	27
4.3	Diagramme de déploiement	27
4.3.1	Environnements de déploiement	27
4.3.2	Configuration Docker Compose locale	28
4.3.3	Specifications techniques minimales	29
4.3.4	Strategie de déploiement en production	31
III	Conception des Modules Metier	32
5	Modules du Coeur Metier Initial	33
5.1	Module Authentification — Diagramme de sequence	33
5.1.1	Flux d'authentification detaille	33
5.1.2	Securite du token JWT	33
5.1.3	Gestion des sessions et revocation	34
5.1.4	Mecanisme de rafraichissement de token	36
5.2	Module Vehicule — Machine a etats	36
5.3	Module Trajet — Diagramme d'etats	37
5.3.1	Etats du trajet et transitions	37
5.3.2	Regles metier des transitions	37
5.3.3	Calculs automatises lors de la completion	38
5.3.4	Persistence de la telemetrie GPS	38
5.4	Module Georeperage — Diagramme de sequence	40
5.4.1	Architecture du module de georeperage	40
5.4.2	Requete PostGIS de detection	40
5.4.3	Algorithme de dedoublonnage des alertes	41
5.4.4	Performances et optimisations	43
6	Module Operations Terrain	44
6.1	Machine a etats de l'Incident	44
6.1.1	Cycle de vie complet d'un incident	44
6.1.2	Regles metier et invariants	44
6.1.3	Methodes metier de l'entite Incident	45
6.1.4	Statistiques et KPIs des incidents	46
6.2	Diagramme de sequence — Declaration d'un incident	48
6.2.1	Flux de declaration d'incident	48
6.2.2	Regles de validation	48

6.2.3	Gestion des photos d'incident	51
7	Module Planification et Affectations	53
7.1	Machine a etats du Planning	53
7.1.1	Etats et transitions du planning	53
7.1.2	Regles de modification selon l'etat	54
7.1.3	Validation avant publication	54
7.2	Detection de conflits — Diagramme de sequence	55
7.2.1	Types de conflits detectes	55
7.2.2	Algorithme de detection optimise	55
7.2.3	Gestion des chevauchements temporels	57
7.2.4	Optimisations de performance	59
7.3	Algorithme de detection de conflits	60
8	Module Documents Legaux et KPIs	61
8.1	Job nocturne des documents — Diagramme de sequence	61
8.1.1	Architecture du job de verification	61
8.1.2	Documents critiques et blocage automatique	62
8.1.3	Frequence des notifications par niveau d'urgence	62
8.1.4	Strategie de renouvellement proactif	65
8.2	Calcul des KPIs — Diagramme d'activite	66
8.2.1	Architecture du module KPI	66
8.2.2	Algorithme de calcul parallelise	67

Table des figures

2.1	Diagramme de cas d'utilisation global de FleetMan. Les quatre acteurs (Super Administrateur, Administrateur, Gestionnaire et Conducteur) interagissent avec 35 cas d'utilisation repartis en 9 domaines fonctionnels. Le Gestionnaire de Flotte dispose du perimetre fonctionnel le plus large avec acces a 22 cas d'utilisation, tandis que le Conducteur est limite aux operations terrain (8 cas d'utilisation).	9
3.1	Diagramme de packages de FleetMan. Le package domain (en vert) ne contient aucune annotation Spring ni aucune dependance externe ; le package application (en bleu) definit les interfaces de ports et les services orchestrateurs ; le package infrastructure (en orange) contient tous les adapters techniques (REST, R2DBC, Kafka, Redis, services externes). Les fleches en tirets representent des relations d'utilisation, toujours orientees vers le domaine (regle de dependance).	15
3.2	Diagramme de composants de FleetMan. Les composants internes (controllers, services, adapters) sont representes en bleu et vert ; les bases de donnees et services externes en gris et jaune. Les fleches pleines indiquent le sens des appels synchrones ; les fleches tiretees representent les flux asynchrones (Kafka, Redis pub/sub). Les composants sont regroupes par couche hexagonale : infrastructure (inbound et outbound), application, et domaine.	18
4.1	Diagramme de classes de la couche domaine. Les records Java immuables (Vehicle, Fleet, Driver, Trip) sont distingues des classes avec etat mutable (Maintenance, Incident, Assignment). Les Value Objects (Coordinates, Address, Contact) sont representes avec le stereotype «ValueObject». Les enumerations (VehicleStatus, MaintenanceType, IncidentSeverity) definissent les etats autorises. Les relations de composition (losange noir) indiquent une dependance forte du cycle de vie ; les associations simples (fleches) representent des references par identifiant UUID.	23

4.2	Diagramme de classes des Ports et Adapters. Les interfaces de ports entrants (use cases) sont representees en vert a gauche ; les interfaces de ports sortants en orange a droite. Les implementations de use cases (services) sont en bleu au centre. Les adapters d’infrastructure (en gris) implementent les ports sortants. Les fleches en tirets symbolisent les relations d’implementation («implements»). Les fleches pleines representent les relations d’utilisation («uses»). Cette organisation garantit que le domaine ne depend d’aucune technologie concrete (inversion de dependance).	26
4.3	Diagramme de deploiement de FleetMan en environnement local. Trois conteneurs Docker : PostgreSQL avec extension PostGIS (port 5433), Redis avec politique LRU (port 6380) et Kafka en mode KRaft sans ZooKeeper (port 9092). L’application Spring Boot communique avec PostgreSQL via R2DBC (driver reactif), avec Redis via Lettuce (client reactif) et avec Kafka via Spring Cloud Stream. Les services externes (Auth, Kernel, Maps) sont accedes via HTTPS avec Circuit Breaker Resilience4j. Liquibase s’execute au demarrage de l’application pour appliquer les 10 migrations de schema de facon incrementale.	30
5.1	Diagramme de sequence : Authentification d’un utilisateur. La fleche (1) initie l’appel HTTP POST <code>/api/v1/auth/login</code> avec email et password ; les fleches (2)–(3) illustrent la delegation au service Auth externe via WebClient reactif ; la fleche (4) montre le rapatriement du token JWT RS256 ; les fleches (5)–(6) representent la creation/mise a jour de l’utilisateur local et le cache des permissions dans Redis ; la fleche (7) retourne le token au client. Le diagramme inclut egalement le flux de validation de token pour les requetes subsequentes (partie droite) : extraction du token depuis l’en-tete Authorization, validation de la signature RS256, verification de l’expiration, consultation de la blacklist Redis, et injection du contexte de securite.	35
5.2	Machine a etats du vehicule FleetMan. Le statut AVAILABLE est l’etat initial. Seul un vehicule AVAILABLE peut demarrer un trajet. Un vehicule en MAINTENANCE ne peut pas etre affecte.	36

- 5.3 Diagramme d'états : Cycle de vie d'un Trajet. Les transitions sont déclenchées par les actions du conducteur (`startTrip`, `endTrip`) ou du gestionnaire (`cancel`). Les états `PLANNED` et `READY` sont des états préparatoires sans télémétrie GPS. L'état `ONGOING` est l'état actif durant lequel le conducteur envoie des données de télémétrie toutes les 30 secondes et le géorepérage est actif. L'état `COMPLETED` est terminal et déclenche les calculs automatisés (distance, durée, consommation, score de conduite). L'état `CANCELLED` peut être atteint depuis n'importe quel état non terminal et nécessite un motif obligatoire. . . . 39
- 5.4 Diagramme de séquence : Envoi de télémétrie et alerte de géorepérage. Les flèches pleines représentent des appels synchrones réactifs (`Mono/Flux`) ; les flèches tiretées représentent des messages asynchrones Kafka en mode `fire-and-forget`. Après réception de la télémétrie (1), l'application valide le format GPS et l'appartenance du véhicule au tenant ; stocke les coordonnées GPS dans Redis (2) avec clef `vehicle:position:<vehicleId>` et TTL de 5 minutes pour un accès temps réel ; persiste le point décimé en PostgreSQL si intervalle ≥ 2 minutes depuis dernier stockage (3) ; publie l'événement `TelemetryReceived` sur Kafka (4). Le consommateur géofence (service réactif autonome) reçoit l'événement, exécute la requête PostGIS de détection de violations (5), consulte le cache de dédoublement Redis (6), et envoie une notification push via l'adaptateur externe (7) si violation détectée et non dédoublement. Ce flux asynchrone garantit que l'envoi de télémétrie par le conducteur retourne immédiatement (latence $< 50\text{ms}$) sans être bloqué par les traitements lourds (détection géofence, notifications). 42
- 6.1 Diagramme d'états : Cycle de vie d'un Incident. L'état `CLOSED` est terminal : aucune modification n'est possible après clôture (immuabilité pour l'audit). La transition vers `RESOLVED` horodate automatiquement le champ `resolvedAt` et nécessite des notes de résolution obligatoires. La méthode métier `isCritical()` retourne `true` si la sévérité est `HIGH` ou `CRITICAL`, déclenchant une notification prioritaire au gestionnaire et un blocage automatique du véhicule. L'état `IN_PROGRESS` permet les mises à jour itératives (ajout de photos, commentaires, pièces commandées) sans changer d'état. La transition `RESOLVED` $\rightarrow$ `IN_PROGRESS` est autorisée en cas de régression (incident résolu mais problème réapparu). 47

- 6.2 Diagramme de sequence : Declaration d'un incident terrain. Le service verifie l'existence du vehicule et du conducteur avant de persister l'incident et de publier l'evenement Kafka. La creation d'un incident suit un flux en 7 etapes : (1) Verification du vehicule via **VehiclePersistencePort**, avec consultation du cache Redis en premier pour limiter les requetes PostgreSQL ; (2) Verification de l'affectation conducteur-vehicule via requete SQL sur **assignments** avec filtre sur statut **IN_PROGRESS** et chevauchement temporel ; (3) Construction de l'entite **Incident** avec validation des invariants dans le constructeur compact (record) ou explicite (classe) ; (4) Persistance en base de donnees via **IncidentPersistencePort** implementee par **IncidentR2dbcAdapter** ; (5) Si severite **HIGH** ou **CRITICAL**, mise a jour atomique du statut vehicule en **MAINTENANCE** (meme transaction) ; (6) Publication d'un evenement **IncidentCreated** sur Kafka en mode fire-and-forget avec gestion d'erreur silencieuse ; (7) Retour de l'entite persistee au client avec HTTP 201 Created et header Location contenant l'URI de la ressource. Ce dernier point garantit que l'echec de la notification ne fait pas echouer la creation de l'incident (resilience). 50
- 7.1 Diagramme d'etats : Cycle de vie d'un Planning. En etat **DRAFT**, le gestionnaire peut modifier librement les affectations sans validation de conflits (edition rapide). La publication declenche une validation complete (conflits, disponibilites, documents) et rend le planning visible aux conducteurs. L'etat **ACTIVE** est atteint automatiquement lorsque la date de debut du planning est atteinte (transition declenchee par job planifie quotidien). L'etat **COMPLETED** est egalement atteint automatiquement lorsque toutes les affectations sont terminees (statut **COMPLETED** ou **CANCELLED**). La transition **PUBLISHED** → **DRAFT** permet au gestionnaire de depublier le planning pour effectuer des modifications importantes (les conducteurs ne voient plus le planning pendant l'edition). La transition **ACTIVE** → **COMPLETED** necessite que 100% des affectations soient dans un etat terminal (**COMPLETED** ou **CANCELLED**). 56

7.2	Diagramme de sequence : Creation d'affectation avec detection de conflits.	
	Les deux requetes SQL de detection utilisent des index partiels sur les statuts actifs (PENDING, IN_PROGRESS), garantissant une complexite en $O(\log n)$. Le flux complet comprend 9 etapes : (1) Reception de la requete HTTP POST avec payload JSON contenant vehicleId, driverId, startDatetime, endDatetime ; (2) Validation du vehicule : consultation cache Redis puis fallback PostgreSQL si cache miss ; (3) Verification du statut vehicule (doit etre AVAILABLE, pas MAINTENANCE ni ON_TRIP) ; (4) Validation du conducteur : existence et statut ACTIVE ; (5) Detection conflits vehicule via requete SQL avec index partiel sur (vehicle_id, start_datetime, end_datetime) WHERE status IN ('PENDING', 'IN_PROGRESS') ; (6) Detection conflits conducteur via requete similaire sur driver_id ; (7) Verification maintenances planifiees du vehicule sur la periode ; (8) Verification indisponibilites declarees du conducteur ; (9) Si aucun conflit, persistance de l'affectation avec statut PENDING et publication evenement Kafka pour notification au conducteur. En cas de conflit a n'importe quelle etape, une exception <code>PlanningException</code> est levee avec code erreur specifique (PLN-006 pour conflit vehicule, PLN-007 pour conflit conducteur, PLN-008 pour maintenance, PLN-009 pour indisponibilite), mappee en HTTP 409 Conflict avec details JSON du conflit.	58

8.1 Diagramme de sequence : Job nocturne de verification des documents.

Le job s'exécute chaque nuit à 2h00 (`@Scheduled(cron="0 0 2 * *")`) et traite en reactif l'ensemble des documents proches de leur expiration ou déjà expirés. Le flux complet comprend : (1) Recupération des documents via requête R2DBC reactive filtrée par `expiryDate <= today + 30 days` ; (2) Transformation du Flux en stream reactif `Flux<Document>` pour traitement pipeline ; (3) Classification par niveau d'urgence (`EXPIRED`, `CRITICAL_7`, `WARNING_15`, `INFO_30`) via opérateur `.map()` ; (4) Génération des alertes via opérateur `.flatMap()` avec persistance dans `document_alerts` ; (5) Publication événements Kafka en batch via `.buffer(50)` pour optimiser le throughput ; (6) Blocage automatique des véhicules/conducteurs avec documents expirés critiques via `.filter(d -> d.isCritical())` puis mise à jour statut en transaction ; (7) Log des statistiques (nombre documents traités, alertes générées, blocages effectués) pour monitoring. Le traitement est asynchrone et non bloquant (`.subscribe()`) pour ne pas bloquer l'application. En cas d'erreur sur un document, le traitement continue pour les autres (resilience via `.onErrorContinue()`). Les métriques Prometheus sont exposées : `fleetman_documents_expired_total`, `fleetman_documents_alerts_sent_tot`

Liste des tableaux

1.1	Comparaison des solutions de gestion de flotte existantes	5
2.1	Acteurs du systeme FleetMan et leurs responsabilites	6
2.2	Matrice de tracabilite acteurs vs domaines fonctionnels	10
2.3	Exigences non fonctionnelles de la plateforme FleetMan	11
3.1	Comparaison Spring MVC vs Spring WebFlux pour FleetMan	13
3.2	Services externes et patterns d'integration	19
4.1	Specifications techniques par environnement	29
7.1	Matrice des operations autorisees par etat de planning	54
8.1	Documents critiques et actions automatiques	62

Resume

Contexte et objectif. La gestion efficace d’une flotte de vehicules constitue un enjeu operationnel et economique majeur pour les entreprises de transport en Afrique centrale. Ce projet vise a concevoir et implementer **FleetMan**, une plateforme B2B de gestion de flotte complete, deploye comme module metier de l’ecosysteme RT-Comops.

Materiel et methodes. Le backend est developpe en Java 21 sur Spring Boot 3.2 avec une architecture hexagonale reactive (WebFlux, R2DBC, PostgreSQL 16). Le frontend est realise en Next.js 16 avec TypeScript et Tailwind CSS. La methodologie adoptee est le Domain-Driven Design associe au pattern Ports & Adapters.

Resultats. La plateforme expose **134 endpoints REST** documentes via Swagger/OpenAPI, repartis en **20 modules fonctionnels**. Dix migrations Liquibase structurent **35 tables PostgreSQL**. La latence P95 cible est inferieure a 200 ms.

Discussion et conclusion. FleetMan constitue une reponse complete aux besoins de supervision de flotte identifies. Son architecture hexagonale garantit une testabilite maximale et une extractibilite vers des microservices sans refactorisation du domaine.

Mots-cles : gestion de flotte, architecture hexagonale, Spring WebFlux, multi-tenant, georeperage, Domain-Driven Design, R2DBC.

Introduction Generale

Contexte general

Le secteur du transport routier assure plus de 80 % du déplacement des personnes et des marchandises en Afrique subsaharienne (**afdb2022transport**). Selon l'Union Internationale des Transports Routiers, le poste carburant represente entre 30 % et 40 % des charges d'exploitation d'un transporteur (**iru2023costs**).

Contexte specifique

Au Cameroun, la majorite des entreprises de transport gerent encore leurs flottes de facon manuelle : carnets papier, appels telephoniques pour localiser les vehicules, absence de tracabilite des incidents. Cette situation engendre :

- des pannes non anticipees dues a l'absence de maintenance preventive ;
- une consommation carburant non maitrisee ;
- une non-conformite documentaire exposant l'entreprise a des amendes ;
- l'impossibilite de calculer le cout reel d'exploitation par vehicule.

Problematique

Comment concevoir et implementer une plateforme numerique de gestion de flotte adaptee aux besoins des entreprises de transport africaines, garantissant le suivi en temps reel des vehicules, la tracabilite des operations terrain et la maitrise des couts d'exploitation, tout en s'integrant harmonieusement dans un ecosysteme logiciel multi-tenant existant ?

Motivation et objectifs SMART

1. **Specifique** : Developper un backend REST et un frontend web couvrant 20 modules fonctionnels de gestion de flotte.
2. **Mesurable** : Exposer au minimum 100 endpoints REST documentes.

3. **Atteignable** : Technologies maitrisees par l'equipe (Spring Boot, React, PostgreSQL).
4. **Realiste** : Projet de fin d'etudes de 5^{eme} annee, developpe sur 6 mois.
5. **Temporel** : Livraison finale en juin 2026.

Structure du rapport

Ce rapport est organise en cinq parties : **(I)** Etat de l'art et analyse de l'existant ; **(II)** Analyse des besoins ; **(III)** Conception de l'architecture ; **(IV)** Implementation et resultats ; **(V)** Integration au Kernel RT-Comops et perspectives.

Première partie

Etat de l'Art et Analyse de l'Existant

Chapitre 1

Etat de l'Art des Solutions de Gestion de Flotte

1.1 Justification des criteres d'evaluation

Avant de comparer les solutions, nous justifions les cinq criteres retenus pour orienter la comparaison vers les besoins reels du projet :

1. **Adapte au contexte africain** : connectivite intermittente, devise XAF, support FR/EN.
2. **Architecture reactive** : flux GPS continu a haute frequence.
3. **Multi-tenancy natif** : isolation stricte entre entreprises.
4. **Modules operations terrain** : maintenance, incidents, carburant — postes de cout principaux.
5. **Open source et integrabilite** : perennite et integration dans l'ecosysteme RT-Comops.

Le tableau 1.1 presente la comparaison issue de cette analyse.

Tab. 1.1 : Comparaison des solutions de gestion de flotte existantes

Critere	Samsara	Geotab	Traccar	Fleetstack	OpenGTS	FleetMan
Contexte africain	Non	Non	Partiel	Partiel	Non	Oui
Architecture reactive	Partiel	Non	Non	Non	Non	Oui
Multi-tenancy natif	Oui	Oui	Partiel	Non	Non	Oui
Operations terrain	Partiel	Partiel	Non	Non	Non	Oui
Open source	Non	Non	Oui	Oui	Oui	Oui
Integration Kernel	Non	Non	Non	Non	Non	Oui

1.2 Positionnement de FleetMan

FleetMan se distingue par son architecture hexagonale reactive, son integration native dans l'écosysteme RT-Comops, et ses modules d'operations terrain complets. Il constitue la premiere solution de gestion de flotte integree dans un ecosysteme ERP multi-tenant pour le marche camerounais.

Chapitre 2

Analyse des Besoins

2.1 Acteurs du systeme

Tab. 2.1 : Acteurs du systeme FleetMan et leurs responsabilites

Acteur	Role	Responsabilites
Super Administrateur	SUPER_ADMIN	Gestion globale de la plateforme, creation des administrateurs
Administrateur	FLEET_ADMIN	Supervision des gestionnaires, statistiques globales
Gestionnaire de Flotte	FLEET_MANAGER	Gestion quotidienne de la flotte, supervision des conducteurs
Conducteur	FLEET_DRIVER	Demarrage des trajets, declaration d'incidents et maintenances

2.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (figure 2.1) presente l'ensemble des interactions entre les quatre acteurs et les fonctionnalites du systeme, regroupees en huit domaines fonctionnels : authentication, vehicules, conducteurs, trajets, operations terrain, geo-reperage, planification, documents et KPIs.

2.2.1 Analyse detaillee des cas d'utilisation

Le systeme FleetMan compte **35 cas d'utilisation** repartis en **9 domaines fonctionnels**. Cette organisation modulaire repond aux principes de cohesion forte et couplage faible preconises par le Domain-Driven Design.

Domaine Authentication et Autorisation

Ce domaine comprend 4 cas d'utilisation fondamentaux :

- **S’authentifier** : Verification des identifiants via le service Auth externe, generation de token JWT RS256 avec duree de vie de 24h.
- **Gerer les permissions** : Attribution granulaire de permissions par role (RBAC) avec 45 permissions distinctes.
- **Consulter son profil** : Acces en lecture seule aux informations personnelles synchronisees depuis le Kernel.
- **Gerer les sessions** : Consultation et revocation des sessions actives pour renforcer la securite.

Domaine Vehicules

Le domaine vehicules constitue le coeur du systeme avec 8 cas d’utilisation :

- **Enregistrer un vehicule** : Saisie complete des caracteristiques techniques (marque, modele, immatriculation, capacite, motorisation).
- **Consulter la liste des vehicules** : Filtrage avance par statut, type, flotte, et recherche textuelle.
- **Modifier un vehicule** : Mise a jour des informations hors immatriculation (immuable apres creation).
- **Archiver un vehicule** : Soft-delete preservant l’historique.
- **Consulter l’historique complet** : Trajets, maintenances, incidents et consommations sur une periode donnee.
- **Exporter les donnees vehicules** : Generation CSV/Excel pour analyse externe.
- **Gerer les documents vehicules** : Upload, consultation et suivi d’expiration (assurance, visite technique, carte grise).
- **Calculer les couts d’exploitation** : Agregation automatique carburant + maintenance + amortissement.

Domaine Conducteurs

Ce domaine comprend 6 cas d’utilisation :

- **Enregistrer un conducteur** : Saisie identite, contact, permis de conduire, photo.

- **Consulter la liste des conducteurs** : Filtrage par statut (ACTIVE, IN-ACTIVE, ON_LEAVE) et flotte.
- **Evaluer les performances** : Consultation du score de conduite, classement, historique des trajets.
- **Gerer les documents conducteurs** : Permis, certificat medical, casier judiciaire avec alertes d'expiration.
- **Desactiver temporairement** : Suspension en cas d'incident grave ou expiration de documents obligatoires.
- **Consulter le planning personnel** : Le conducteur voit ses affectations futures et peut signaler une indisponibilite.

Domaine Trajets et Telemetrie

Ce domaine gere le cycle de vie des missions avec 7 cas d'utilisation :

- **Creer un trajet** : Definition du point de depart, destination, heure prevue, distance estimee.
- **Demarrer un trajet** : Validation de l'etat du vehicule, activation du tracking GPS.
- **Envoyer la telemetrie GPS** : Transmission temps reel toutes les 30 secondes (position, vitesse, cap).
- **Terminer un trajet** : Calcul automatique distance reelle, duree, consommation estimee.
- **Consulter l'historique des trajets** : Filtrage par vehicule, conducteur, periode avec export PDF.
- **Visualiser un trajet sur carte** : Replay de l'itineraire complet avec points d'interet.
- **Comparer trajet prevu vs reel** : Analyse des ecarts de distance, duree et cout.

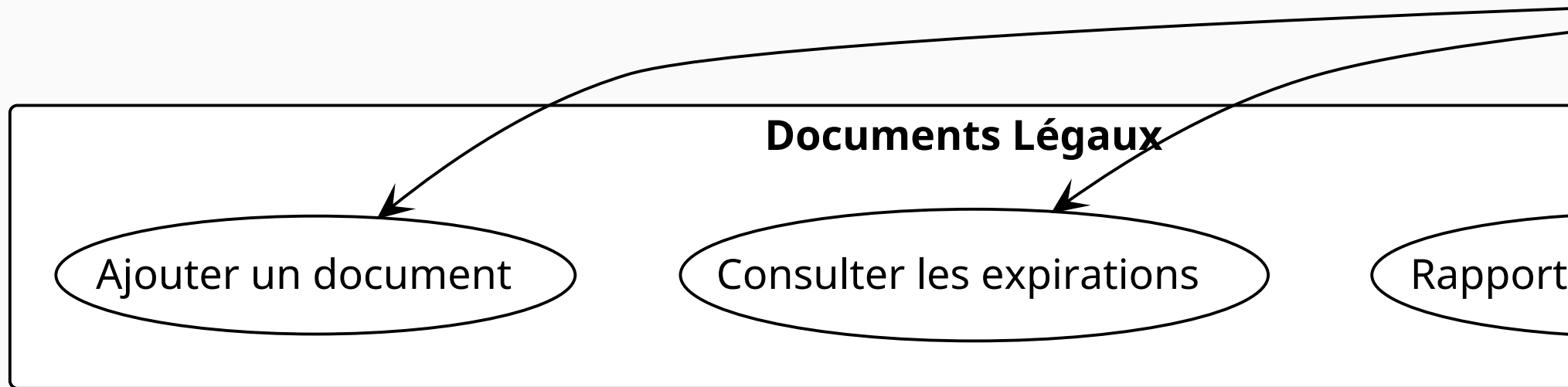


Fig. 2.1 : Diagramme de cas d'utilisation global de FleetMan. Les quatre acteurs (Super Administrateur, Administrateur, Gestionnaire et Conducteur) interagissent avec 35 cas d'utilisation repartis en 9 domaines fonctionnels. Le Gestionnaire de Flotte dispose du périmètre fonctionnel le plus large avec accès à 22 cas d'utilisation, tandis que le Conducteur est limité aux opérations terrain (8 cas d'utilisation).

Lecture du diagramme

Chaque rectangle gris represente un domaine fonctionnel du systeme. Les fleches pleines symbolisent les associations acteur-cas d'utilisation. Le Gestionnaire de Flotte est l'acteur principal : il interagit avec la majorite des fonctionnalites (22 cas sur 35). Le Conducteur est limite aux operations de terrain et au suivi de ses propres affectations. Le Super Administrateur dispose de privileges transverses pour la gestion multi-tenant et la configuration globale de la plateforme. La hierarchie des permissions suit le principe RBAC (Role-Based Access Control) avec heritage : SUPER_ADMIN herite de FLEET_ADMIN qui herite de FLEET_MANAGER qui herite de FLEET_DRIVER.

2.2.2 Matrice de tracabilite acteurs-fonctionnalites

Tab. 2.2 : Matrice de tracabilite acteurs vs domaines fonctionnels

Domaine	Super Admin	Admin	Manager	Driver
Authentification	Oui	Oui	Oui	Oui
Vehicules	Lecture	Complet	Complet	Lecture
Conducteurs	Lecture	Complet	Complet	Profil
Trajets	Non	Lecture	Complet	Execution
Georeperage	Non	Lecture	Complet	Non
Operations	Non	Lecture	Complet	Declaration
Planification	Non	Lecture	Complet	Consultation
Documents	Non	Lecture	Complet	Upload
KPIs	Global	Flotte	Flotte	Non
Administration	Complet	Partiel	Non	Non

2.3 Exigences non fonctionnelles

Tab. 2.3 : Exigences non fonctionnelles de la plateforme FleetMan

Code	Categorie	Exigence
ENF-01	Performance	Latence P95 ≤ 200 ms (100 utilisateurs simultanes)
ENF-02	Disponibilite	Disponibilite $\geq 99,5\%$ hors maintenance
ENF-03	Temps reel	Latence GPS ≤ 5 s entre position reelle et affichage
ENF-04	Securite	Chiffrement TLS 1.3 en transit, AES-256 au repos
ENF-05	Scalabilite	Architecture stateless permettant le scaling horizontal
ENF-06	Resilience	Circuit Breaker sur tous les appels aux services externes
ENF-07	Observabilite	Metriques Prometheus, logs structures JSON
ENF-08	Maintenabilite	Couverture de tests $\geq 70\%$ sur la couche domaine

Deuxième partie

Conception de l'Architecture

Chapitre 3

Choix Technologiques et Architecture Hexagonale

3.1 Justification des choix technologiques

Tab. 3.1 : Comparaison Spring MVC vs Spring WebFlux pour FleetMan

Critere	Spring MVC	Spring WebFlux
Modele de concurrence	Thread par requete	Evenementiel (Netty)
Connexions simultanees	~200 (pool threads)	>10 000
Integration R2DBC	Non	Oui (natif)
Adapte aux flux GPS	Non	Oui
Choix pour FleetMan	Rejete	Retenu

Spring WebFlux a ete retenu car la gestion de flotte implique des flux de donnees continus (GPS, alertes geofence) qu'un modele bloquant ne peut pas absorber efficacement. PostgreSQL avec R2DBC a ete prefere a MongoDB pour les donnees hautement relationnelles (vehicule → conducteur → trajets).

3.2 Architecture hexagonale — Vue d'ensemble

L'architecture hexagonale organise le systeme en trois couches concentriques ou les dependances ne pointent que vers l'interieur. La couche Domaine (entites, use cases, ports) est completement isolee de toute dependance technique.

3.3 Diagramme de packages

Le diagramme de la figure 3.1 presente l'organisation complete des packages Java du projet, illustrant la separation stricte entre les couches domaine, application et infrastructure.

3.3.1 Justification de l'architecture en packages

L'organisation en packages suit rigoureusement les principes de l'architecture hexagonale :

1. **Package domain** : Contient les entites metier, Value Objects, enumerations et exceptions metier. Aucune dependance externe (ni Spring, ni Jackson, ni JPA). Ce package est testable unitairement sans contexte Spring.
2. **Package application** : Definit les ports entrants (use cases) et sortants (persistence, messaging, external services). Les interfaces sont du Java pur. Les implementations des use cases (services) resident egalement ici et orchestrent la logique applicative.
3. **Package infrastructure** : Contient tous les adapters implementant les ports sortants, plus les adapters entrants (REST controllers, jobs planifies, event listeners Kafka). C'est le seul package autorisant les annotations Spring, Jackson, R2DBC.

3.3.2 Avantages de cette organisation

- **Testabilite** : Le domaine peut etre teste sans Spring Boot Test (tests rapides).
- **Portabilite** : Le domaine peut etre extrait vers un autre framework (Quarkus, Micronaut) sans modification.
- **Comprehension** : Les nouveaux developpeurs identifient rapidement le coeur metier (domaine) vs les details techniques (infrastructure).
- **Evolutivite** : Remplacer PostgreSQL par MongoDB necessite uniquement de reimplementer les PersistencePorts, sans toucher au domaine.

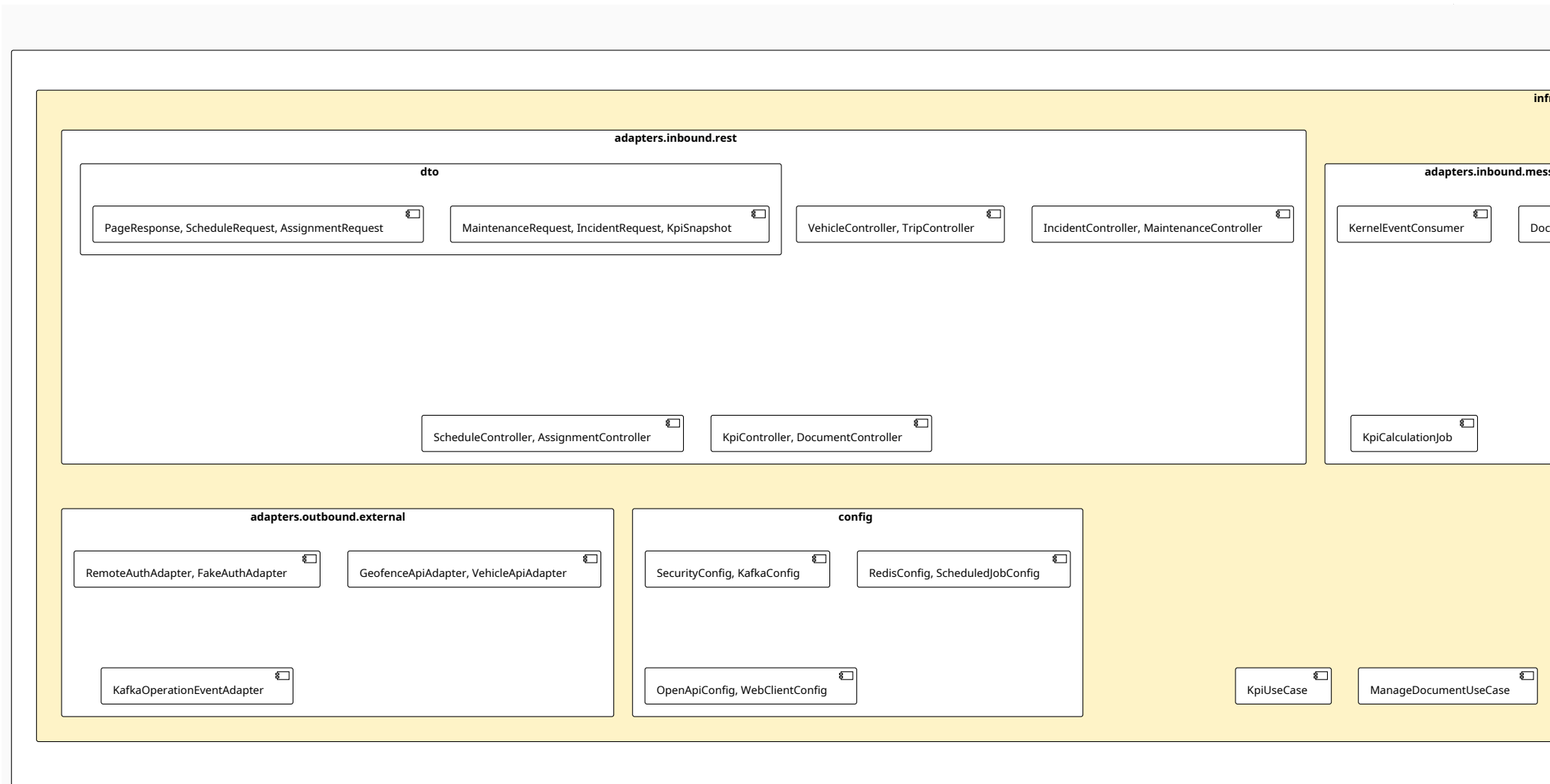


Fig. 3.1 : Diagramme de packages de FleetMan. Le package **domain** (en vert) ne contient aucune annotation Spring ni aucune dépendance externe ; le package **application** (en bleu) définit les interfaces de ports et les services orchestrateurs ; le package **infrastructure** (en orange) contient tous les adaptateurs techniques (REST, R2DBC, Kafka, Redis, services externes). Les flèches en tirets représentent des relations d'utilisation, toujours orientées vers le domaine (règle de dépendance).

Lecture du diagramme

Les fleches en tirets representent des dependances d'utilisation : **infrastructure** utilise **application**, qui utilise **domain**. La regle fondamentale est que **domain** ne depend de rien d'autre que de lui-meme. Cette regle est verifiee automatiquement par ArchUnit dans les tests d'architecture.

3.3.3 Exemple concret de flux inter-packages

Prenons l'exemple d'une creation de vehicule via l'API REST :

1. Le **VehicleController** (infrastructure/adapters/inbound/rest) recoit la requete HTTP POST.
2. Le controller delegue au **ManageVehicleUseCase** (application/ports/inbound).
3. Le service **VehicleService** (application/services) implemente le use case et construit l'entite **Vehicle** (domain/model).
4. Le service appelle **VehiclePersistencePort** (application/ports/outbound).
5. L'adapter **VehicleR2dbcAdapter** (infrastructure/adapters/outbound/persistence) implemente le port et persiste en base PostgreSQL.
6. L'adapter publie un evenement via **OperationEventPort** implemente par **KafkaEventPublisher**.

Cette chaine montre que chaque couche respecte son role sans jamais violer la regle de dependance.

3.4 Diagramme de composants

La figure 3.2 presente l'architecture de composants du systeme, montrant les interactions entre les controllers, services, adapters et services externes.

3.4.1 Composants principaux et responsabilites

L'architecture de FleetMan s'appuie sur 6 categories de composants :

1. **REST Controllers (Inbound Adapters)** : Exposition HTTP des API, validation DTO, mapping requetes/reponses, gestion des erreurs HTTP. Chaque controller est annote `@RestController` et documente via `@Operation` pour Swagger.

2. **Application Services (Use Case Implementations)** : Orchestration de la logique metier, appel des ports sortants, gestion transactionnelle (via `@Transactional` reactif). Les services sont stateless et peuvent etre horizontalement scalables.
3. **Domain Entities et Value Objects** : Modele metier pur, encapsulation des invariants, validation dans les constructeurs. Les entites complexes (Maintenance, Incident) exposent des methodes metier (ex : `resolve()`, `close()`).
4. **Persistence Adapters (Outbound)** : Implementations R2DBC des ports de persistance. Chaque adapter utilise un repository Spring Data R2DBC et un mapper MapStruct pour convertir entites domaine $\leftrightarrow$ entites JPA.
5. **Messaging Adapters (Outbound)** : Publication d'evenements metier sur Kafka en mode fire-and-forget. Le `KafkaEventPublisher` utilise `ReactiveKafkaProducer` de Spring Cloud Stream.
6. **External Services Adapters (Outbound)** : Appels HTTP vers services externes (Auth, Kernel, maps/geocoding). Chaque adapter utilise `WebClient` reactif avec Resilience4j Circuit Breaker pour la resilience.

FleetMan — Diagramme de Composants

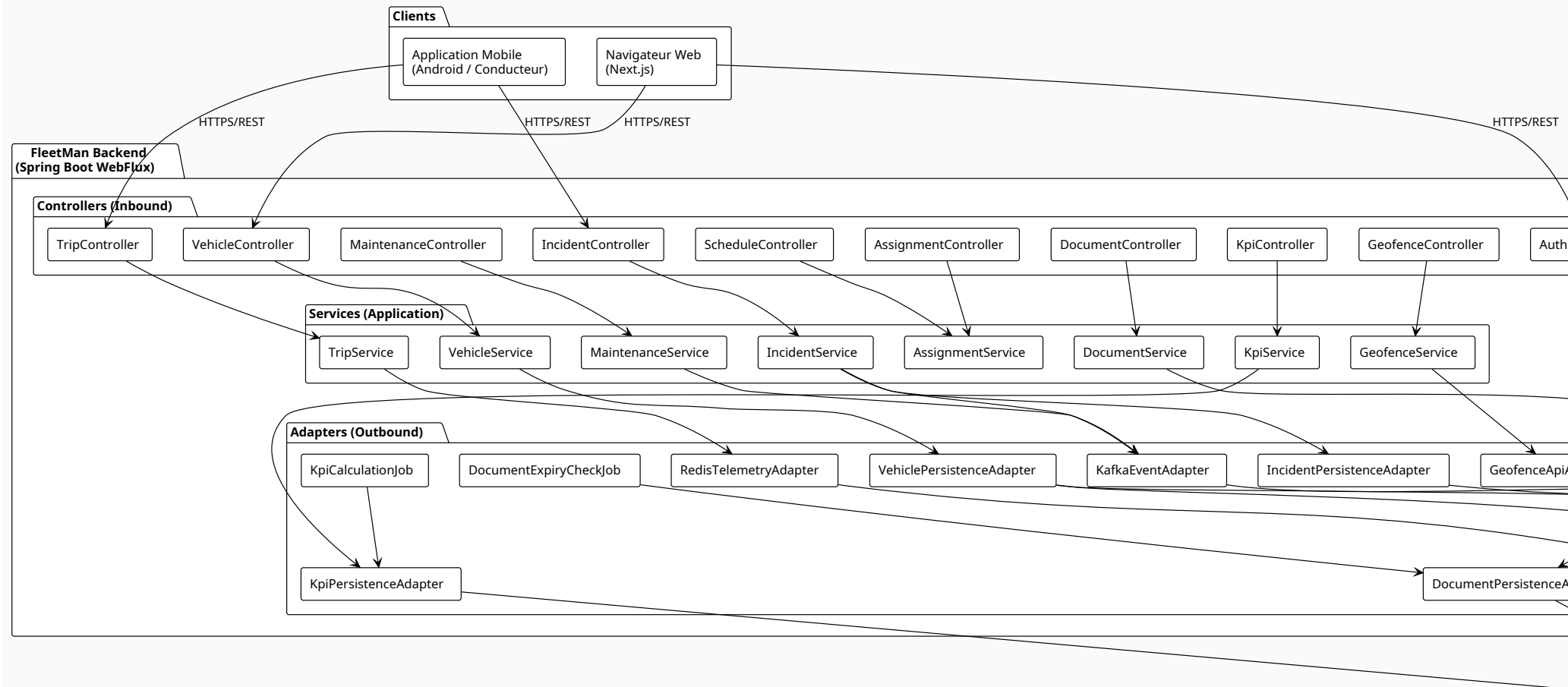


Fig. 3.2 : Diagramme de composants de FleetMan. Les composants internes (contrôleurs, services, adaptateurs) sont représentés en bleu et vert ; les bases de données et services externes en gris et jaune. Les flèches pleines indiquent le sens des appels synchrones ; les flèches tiretées représentent les flux asynchrones (Kafka, Redis pub/sub). Les composants sont regroupés par couche hexagonale : infrastructure (inbound et outbound), application, et domaine.

Lecture du diagramme

Les controllers (inbound) reçoivent les requêtes HTTP et delegent aux services. Les services orchestrent la logique métier et deleguent la persistance aux adapters (outbound). Les jobs planifiés s'exécutent de façon autonome et utilisent les mêmes adapters que les services. Le composant Redis est utilisé pour deux usages distincts : (1) cache des positions GPS temps réel avec TTL de 5 minutes ; (2) cache des permissions utilisateur pour éviter des appels répétitifs au service Auth. Le composant Kafka sert de bus d'événements asynchrone pour les notifications, l'audit et l'intégration avec d'autres modules RT-Comops.

3.4.2 Patterns d'intégration avec services externes

FleetMan intègre 4 services externes via des adapters dédiés :

Tab. 3.2 : Services externes et patterns d'intégration

Service	Pattern	Usage
Auth Service	Circuit Breaker	Validation JWT, récup claims utilisateur
Kernel Core	Circuit Breaker + Cache	Consultation organisations, ressources
Maps/Geocoding	Circuit Breaker + Fallback	Geocodage adresses, calcul itinéraires
Notification Service	Fire-and-Forget (Kafka)	Envoi SMS/Email/Push aux conducteurs

Le Circuit Breaker Resilience4j est configuré avec les seuils suivants :

- **Failure rate threshold** : 50 % (ouverture du circuit).
- **Wait duration in open state** : 60 secondes.
- **Sliding window size** : 10 requêtes.
- **Fallback** : Réponse dégradée (ex : adresse non géocodée).

Chapitre 4

Modele de Donnees et Classes Me- tier

4.1 Diagramme de classes — Couche domaine

La figure 4.1 presente les entites metier du domaine et leurs relations. Ces classes sont du Java pur, sans aucune annotation Spring, JPA ou Jackson.

4.1.1 Entites principales et leurs responsabilites

Entite Vehicle

L'entite `Vehicle` est implementee comme un record Java immuable :

```
public record Vehicle(  
    UUID id,  
    String licensePlate,  
    String brand,  
    String model,  
    int year,  
    VehicleType type,  
    VehicleStatus status,  
    UUID fleetId,  
    String tenantId  
) {  
    // Validation dans le constructeur compact  
}
```

Les invariants valides sont :

- `licensePlate` non vide et unique par tenant.
- `year` entre 1900 et annee courante + 1.

- **status** initialise a AVAILABLE par default.

Entite Maintenance

Contrairement a **Vehicle**, **Maintenance** est une classe mutable car elle possede un cycle de vie avec changements d'etat :

```
public class Maintenance {
    private UUID id;
    private UUID vehicleId;
    private MaintenanceType type;
    private MaintenanceStatus status;
    private LocalDateTime scheduledAt;
    private LocalDateTime completedAt;
    private BigDecimal cost;
    private String description;

    public void markAsCompleted(BigDecimal finalCost) {
        if (this.status != SCHEDULED) {
            throw new InvalidStateException();
        }
        this.status = COMPLETED;
        this.completedAt = LocalDateTime.now();
        this.cost = finalCost;
    }
}
```

Les methodes metier (**markAsCompleted**, **cancel**, **reschedule**) encapsulent la logique de transition d'etat et garantissent la coherence des donnees.

Entite Incident

L'entite **Incident** gere le cycle de vie complet d'un incident terrain avec severite, resolution et impact operationnel :

- **Severite** : LOW, MEDIUM, HIGH, CRITICAL.
- **Statuts** : OPEN, IN_PROGRESS, RESOLVED, CLOSED.
- **Methodes metier** : **resolve()**, **close()**, **escalate()**, **isCritical()**.

Un incident CRITICAL bloque automatiquement le vehicule (statut MAINTENANCE) et notifie le gestionnaire en temps reel via Kafka.

Value Object Coordinates

Coordinates est un Value Object immuable qui valide les bornes GPS dans son constructeur :

```
public record Coordinates(double latitude, double longitude) {  
    public Coordinates {  
        if (latitude < -90 || latitude > 90) {  
            throw new InvalidCoordinatesException("Latitude invalide");  
        }  
        if (longitude < -180 || longitude > 180) {  
            throw new InvalidCoordinatesException("Longitude invalide");  
        }  
    }  
  
    public double distanceTo(Coordinates other) {  
        // Formule de Haversine  
    }  
}
```

Ce Value Object est utilise dans **TelemetryData**, **Geofence** et **Trip**.

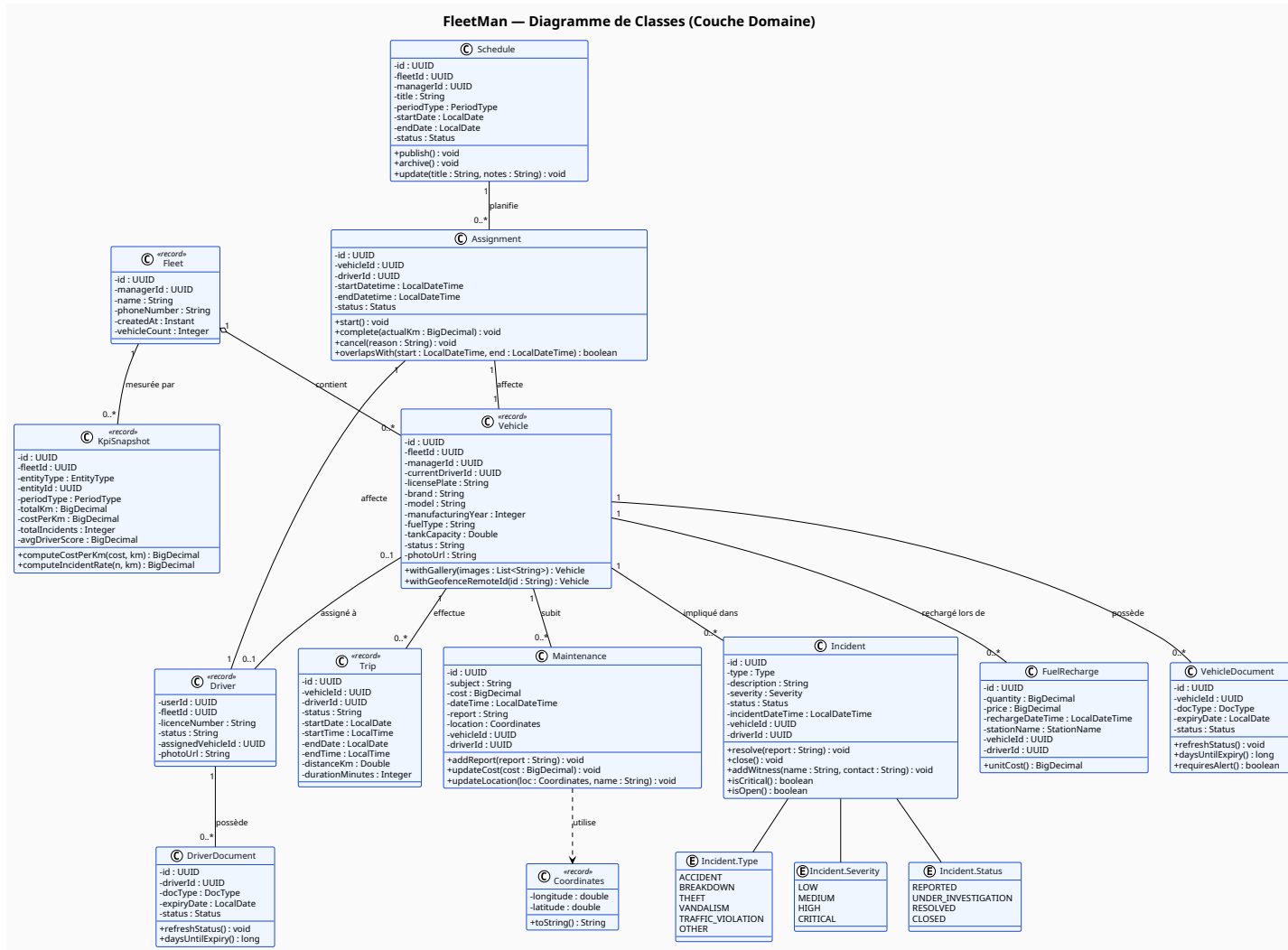


Fig. 4.1 : Diagramme de classes de la couche domaine. Les records Java immuables (Vehicle, Fleet, Driver, Trip) sont distingués des classes avec état mutable (Maintenance, Incident, Assignment). Les Value Objects (Coordinates, Address, Contact) sont représentés avec le stereotype « ValueObject ». Les enumerations (VehicleStatus, MaintenanceType, IncidentSeverity) définissent les états autorisés. Les relations de composition (losange noir) indiquent une dépendance forte du cycle de vie ; les associations simples (flèches) représentent des références par identifiant UUID.

Lecture du diagramme

Les entites **Maintenance**, **Incident** et **FuelRecharge** sont des classes classiques car elles possèdent des methodes metier qui modifient leur etat (ex : `resolve()`, `close()`). Les entites **Vehicle**, **Fleet**, **Driver** et **Trip** sont des records Java immuables car leur modification necessite une nouvelle instance (immutabilite par conception). L'entite **Coordinates** est un Value Object qui valide les bornes GPS dans son constructeur. Les Value Objects sont systematiquement compares par valeur (equals/hashCode sur tous les champs), pas par identite. La relation entre **Vehicle** et **Fleet** est une association simple (reference par UUID), permettant le changement de flotte sans reconstruire le vehicule.

4.1.2 Patterns DDD appliques

Le modele de domaine applique 5 patterns du Domain-Driven Design :

1. **Entity** : Objets avec identite persistante (**Vehicle**, **Driver**, **Trip**, **Maintenance**, **Incident**). L'egalite se fait sur l'ID, pas sur les attributs.
2. **Value Object** : Objets sans identite, compares par valeur (**Coordinates**, **Address**, **Contact**, **Money**). Systematiquement immuables.
3. **Aggregate Root** : Entites racines qui controlent l'accès aux autres entites de l'agregat. Ex : **Trip** est le root de l'agregat [**Trip**, **TelemetryData**]. Toute modification de telemetrie passe par **Trip**.
4. **Factory Method** : Methodes statiques de construction pour les entites complexes. Ex : `Incident.critical()` pour creer un incident de severite **CRITICAL**.
5. **Domain Event** : Evenements metier publies apres operations critiques : **VehicleCreated**, **TripStarted**, **IncidentResolved**. Ces evenements sont publies sur Kafka pour l'integration asynchrone.

4.2 Diagramme de classes — Ports et Adapters

La figure 4.2 illustre la structure des ports et adapters, coeur du pattern hexagonal.

4.2.1 Ports entrants (Inbound Ports / Use Cases)

Les ports entrants definissent ce que le systeme offre a ses utilisateurs. Chaque port correspond a un ensemble de use cases coherents :

- **ManageVehicleUseCase** : CRUD vehicules + historique + statistiques.
- **ManageDriverUseCase** : CRUD conducteurs + evaluation + planning.
- **ManageTripUseCase** : Creation, demarrage, cloture de trajets.
- **ManageIncidentUseCase** : Declaration, suivi, resolution d'incidents.
- **ManageMaintenanceUseCase** : Planification et suivi maintenances.
- **TrackVehicleUseCase** : Reception telemetrie GPS, replay trajets.
- **ManageGeofenceUseCase** : Definition zones, detection violations.
- **PlanScheduleUseCase** : Creation plannings, gestion affectations.
- **ComputeKpiUseCase** : Calcul et consultation indicateurs.

Chaque use case est une interface Java pure, sans dependance Spring :

```
public interface ManageVehicleUseCase {  
    Mono<Vehicle> create(CreateVehicleCommand cmd);  
    Mono<Vehicle> findById(UUID id);  
    Flux<Vehicle> findAll(Pageable pageable);  
    Mono<Vehicle> update(UUID id, UpdateVehicleCommand cmd);  
    Mono<Void> archive(UUID id);  
}
```

4.2.2 Ports sortants (Outbound Ports)

Les ports sortants definissent ce que le domaine requiert de l'infrastructure. Ils sont implements par des adapters dans la couche infrastructure :

- **PersistencePorts** : **VehiclePersistencePort**, **DriverPersistencePort**, **TripPersistencePort**, etc. Chaque port expose des methodes reactives (Mono, Flux) pour les operations CRUD.
- **MessagingPorts** : **OperationEventPort**, **TelemetryEventPort**. Publication asynchrone d'evenements metier sur Kafka.
- **CachePorts** : **TelemetryCachePort**, **PermissionCachePort**. Acces temps reel aux donnees cached dans Redis.
- **ExternalServicePorts** : **AuthServicePort**, **KernelServicePort**, **GeocodingServicePort**. Appels HTTP vers services externes avec Circuit Breaker.

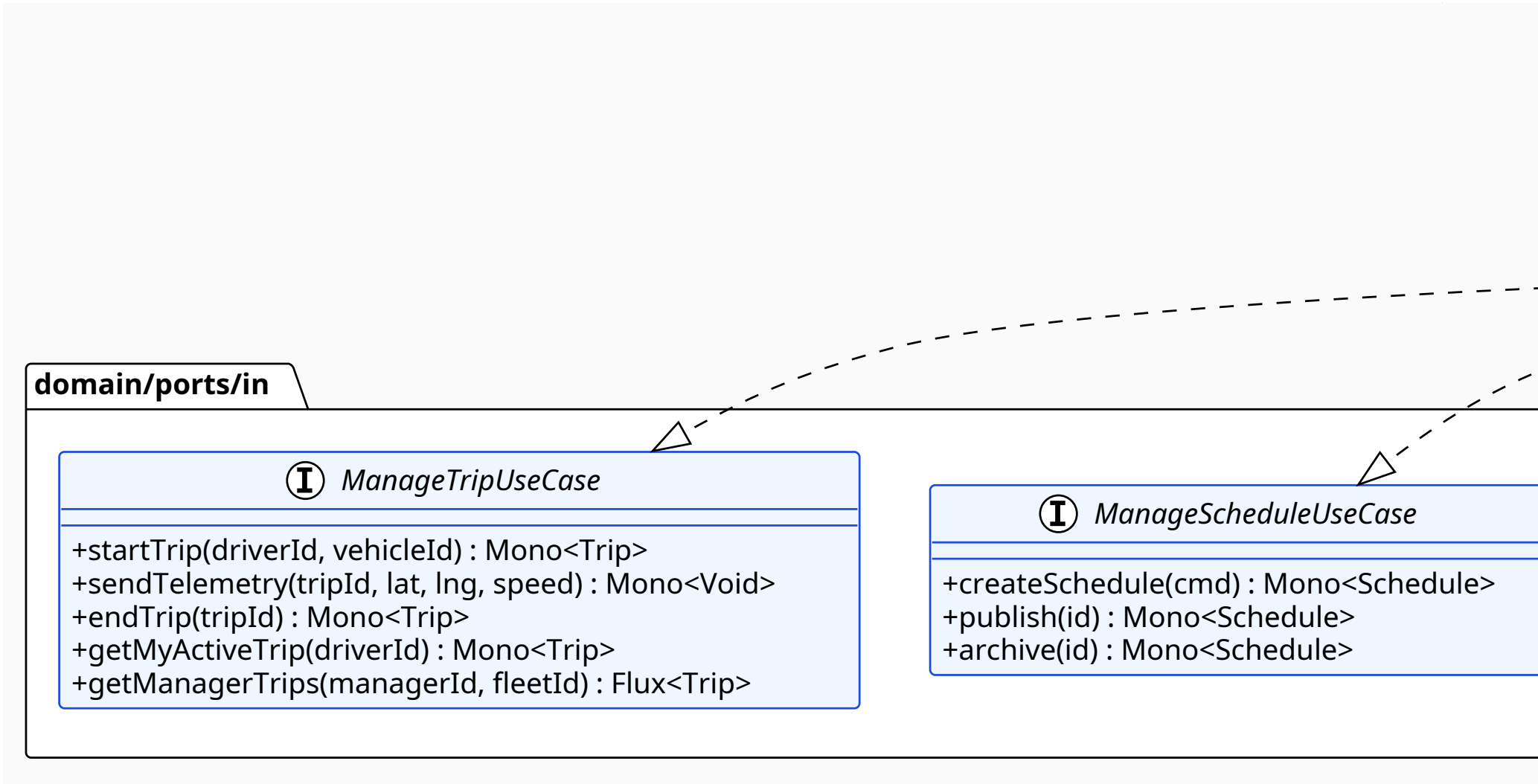


Fig. 4.2 : Diagramme de classes des Ports et Adapters. Les interfaces de ports entrants (use cases) sont représentées en vert à gauche ; les interfaces de ports sortants en orange à droite. Les implémentations de use cases (services) sont en bleu au centre. Les adaptateurs d'infrastructure (en gris) implémentent les ports sortants. Les flèches en tirets symbolisent les relations d'implémentation (« implements »). Les flèches pleines représentent les relations d'utilisation (« uses »). Cette organisation garantit que le domaine ne dépend d'aucune technologie concrète (inversion de dépendance).

Lecture du diagramme

Les interfaces `ManageVehicleUseCase`, `ManageIncidentUseCase`, etc. constituent les ports entrants : ce que le domaine offre. Les interfaces `VehiclePersistencePort`, `OperationEventPort`, etc. constituent les ports sortants : ce que le domaine requiert. Les adapters d'infrastructure implementent les ports sortants sans que le domaine ne les connaisse directement. Cette separation permet de remplacer PostgreSQL par MongoDB ou Kafka par RabbitMQ sans modifier une seule ligne du domaine. Les services (implementations des use cases) orchestrent les ports sortants pour realiser la logique metier.

4.2.3 Avantages du pattern Ports & Adapters

1. **Testabilite** : Les use cases peuvent etre testes unitairement en mockant les ports sortants (pas besoin de base de donnees reelle).
2. **Independance technologique** : Le domaine ne connait ni Spring, ni R2DBC, ni Kafka. Il peut etre extrait vers un autre framework sans refactorisation.
3. **Evolutivite** : Ajouter un nouveau canal d'entree (GraphQL, gRPC) necessite uniquement un nouvel adapter inbound appelant les use cases existants.
4. **Maintenabilite** : La logique metier est centralisee dans les services, pas eparpillee dans les controllers ou repositories.
5. **Migration microservices** : Chaque bounded context (vehicules, conducteurs, trajets) peut etre extrait en microservice independant en packagant son domaine + ses adapters.

4.3 Diagramme de deploiement

La figure 4.3 decrit l'architecture de deploiement en environnement local via Docker Compose.

4.3.1 Environnements de deploiement

FleetMan supporte 4 environnements avec configurations distinctes :

1. **Local (dev)** : Application Spring Boot lancee depuis l'IDE, services externes (PostgreSQL, Redis, Kafka) en conteneurs Docker. Hot-reload active via Spring DevTools.

2. **Integration (int)** : Tous les composants dockerises, deployes via Docker Compose sur machine de CI/CD. Utilise pour les tests d'integration et end-to-end.
3. **Staging (staging)** : Infrastructure Kubernetes (AWS EKS), bases de donnees managees (RDS PostgreSQL, ElastiCache Redis), Kafka MSK. Replication de la production a echelle reduite.
4. **Production (prod)** : Infrastructure Kubernetes avec auto-scaling (HPA), multi-AZ pour haute disponibilite, monitoring Prometheus/Grafana, logs centralises ElasticSearch/Kibana.

4.3.2 Configuration Docker Compose locale

Le fichier `docker-compose.yml` definit 3 services :

```
version: '3.9'
services:
  postgres:
    image: postgis/postgis:16-alpine
    ports: ["5433:5432"]
    environment:
      POSTGRES_DB: fleetman
      POSTGRES_USER: fleet_user
      POSTGRES_PASSWORD: fleet_pass
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports: ["6380:6379"]
    command: redis-server --maxmemory 512mb --maxmemory-policy allkeys-lru

  kafka:
    image: apache/kafka:3.7.0
    ports: ["9092:9092"]
    environment:
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_NODE_ID: 1
      # Mode KRaft (sans ZooKeeper)
```


4.3.3 Specifications techniques minimales

Tab. 4.1 : Specifications techniques par environnement

Ressource	Local	Int	Staging	Production
CPU (cores)	2	4	8	16 (auto-scale)
RAM (GB)	4	8	16	32 (auto-scale)
Disk (GB)	20	50	200	500 (SSD)
PostgreSQL (vCPU)	2	2	4	8 (RDS Multi-AZ)
Redis (GB)	0.5	1	2	4 (ElastiCache)
Kafka (parti- tions)	3	6	12	24 (MSK)
Replicas (pods)	1	2	3	5 min, 20 max

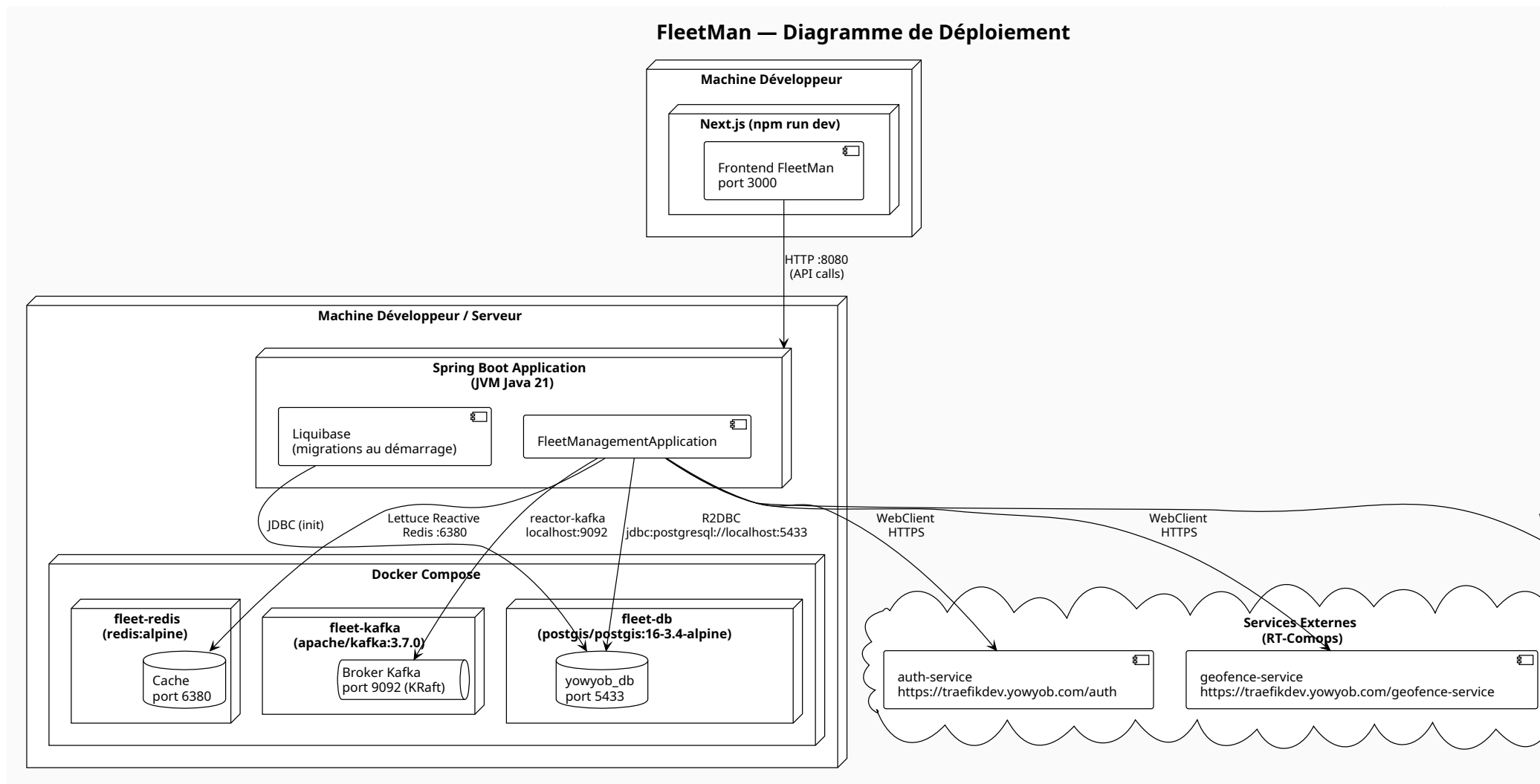


Fig. 4.3 : Diagramme de déploiement de FleetMan en environnement local. Trois conteneurs Docker : PostgreSQL avec extension PostGIS (port 5433), Redis avec politique LRU (port 6380) et Kafka en mode KRaft sans ZooKeeper (port 9092). L'application Spring Boot communique avec PostgreSQL via R2DBC (driver reactif), avec Redis via Lettuce (client reactif) et avec Kafka via Spring Cloud Stream. Les services externes (Auth, Kernel, Maps) sont accedes via HTTPS avec Circuit Breaker Resilience4j. Liquibase s'exécute au démarrage de l'application pour appliquer les 10 migrations de schema de façon incrementale.

Lecture du diagramme

Les boites representent les noeuds de deploiement. L'application Spring Boot est lancee directement sur la JVM Java 21 (hors Docker en developpement). Li-quibase s'execute au demarrage de l'application pour appliquer les migrations de schema de maniere incrementale et idempotente. Les conteneurs Docker sont orchestres via docker-compose.yml pour faciliter la reproductibilite de l'environnement de developpement. En production, l'application est packagee en image Docker multi-stage (build + runtime) et deployee sur Kubernetes avec Helm charts. Le health check endpoint `/actuator/health` est utilise par Kubernetes pour les liveness et readiness probes.

4.3.4 Strategie de deploiement en production

Le deploiement en production suit une strategie Blue-Green avec validation automatique :

1. **Build** : Compilation Maven, tests unitaires + integration, analyse SonarQube, scan vulnerabilites (Trivy).
2. **Package** : Construction image Docker multi-stage, push vers AWS ECR avec tag semantique (semver).
3. **Deploy Green** : Deploiement version N+1 sur environnement Green (parallele au Blue), sans trafic reel.
4. **Smoke Tests** : Execution tests sante (health checks, endpoints critiques, connectivite bases de donnees).
5. **Switch** : Bascule trafic du Blue vers Green via load balancer AWS ALB avec strategie weighted (10% -> 50% -> 100%).
6. **Monitor** : Surveillance metriques (latence P95, taux erreur, CPU/RAM) pendant 15 minutes.
7. **Rollback automatique** : Si taux erreur > 1%, rollback instantane vers version Blue precedente.

Cette strategie garantit un deploiement zero-downtime et un rollback rapide en cas de regression.

Troisième partie

Conception des Modules Metier

Chapitre 5

Modules du Coeur Metier Initial

5.1 Module Authentication — Diagramme de sequence

La figure 5.1 detaille le flux complet d'authentification d'un utilisateur, depuis la soumission des identifiants jusqu'a l'injection du contexte de securite dans la chaine reactive.

5.1.1 Flux d'authentification detaille

Le processus d'authentification FleetMan suit une architecture a 3 niveaux :

1. **Niveau API Gateway** : Reception de la requete HTTP avec credentials (email + password) ou token JWT dans l'en-tete **Authorization: Bearer <token>**.
2. **Niveau Service Auth externe** : Delegation de la validation au service Auth centralisé du Kernel RT-Comops. Ce service valide les credentials, genere un JWT RS256 signe avec cle privee, et retourne les claims utilisateur (userId, tenantId, roles, permissions).
3. **Niveau Application FleetMan** : Stockage ou mise a jour de l'utilisateur local dans la table **users**, cache des permissions dans Redis (TTL 1h), injection du contexte de securite dans le **ReactiveSecurityContextHolder** pour les requetes subsequentes.

5.1.2 Securite du token JWT

Le token JWT utilise l'algorithme RS256 (RSA + SHA-256) avec les caracteristiques suivantes :

- **Cle privee** : 2048 bits, stockee dans AWS Secrets Manager, rotatee tous les 90 jours.

- **Cle publique** : Distribuee aux services via endpoint JWK (`/.well-known/jwks.json`).
- **Claims standards** : `iss` (issuer), `sub` (subject=userId), `iat` (issued at), `exp` (expiration=24h).
- **Claims customs** : `tenantId`, `roles[]`, `permissions[]`, `fleetIds[]`.

5.1.3 Gestion des sessions et revocation

FleetMan maintient une table `user_sessions` pour tracer les sessions actives :

```
CREATE TABLE user_sessions (  
  id UUID PRIMARY KEY,  
  user_id UUID NOT NULL,  
  token_jti VARCHAR(36) NOT NULL UNIQUE,  
  created_at TIMESTAMP NOT NULL,  
  expires_at TIMESTAMP NOT NULL,  
  last_activity TIMESTAMP,  
  ip_address INET,  
  user_agent TEXT,  
  revoked BOOLEAN DEFAULT false  
);
```

La revocation d'une session specifique marque `revoked=true` et ajoute le `token_jti` dans une blacklist Redis (TTL = duree residuelle du token). Toutes les requetes verifient la blacklist avant d'autoriser l'accès.

FleetMan — Diagramme de Séquence : Authentification d'un utilisateur

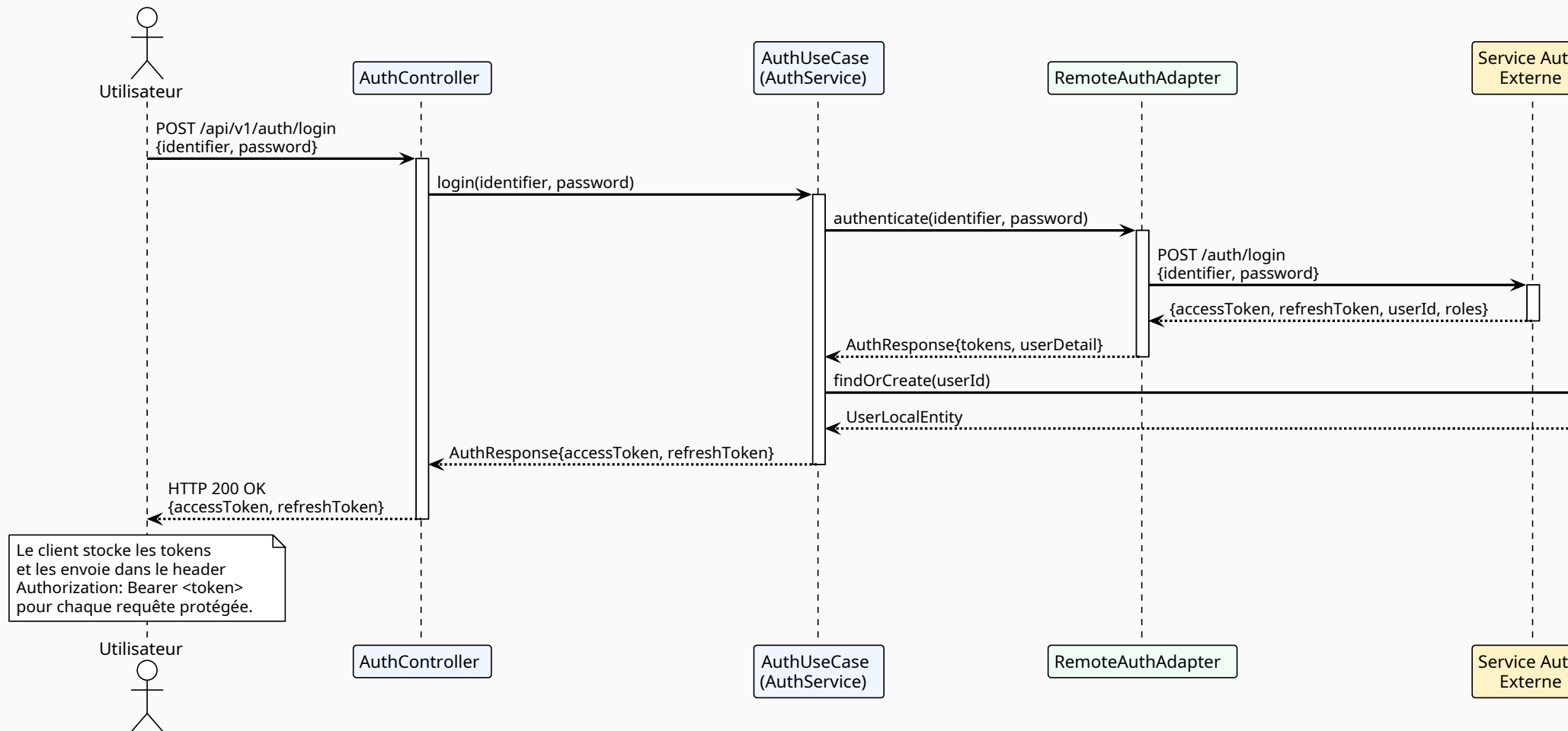


Fig. 5.1 : Diagramme de séquence : Authentification d'un utilisateur. La flèche (1) initie l'appel HTTP POST `/api/v1/auth/login` avec email et password; les flèches (2)–(3) illustrent la délégation au service Auth externe via WebClient réactif; la flèche (4) montre le rapatriement du token JWT RS256; les flèches (5)–(6) représentent la création/mise à jour de l'utilisateur local et le cache des permissions dans Redis; la flèche (7) retourne le token au client. Le diagramme inclut également le flux de validation de token pour les requêtes subséquentes (partie droite) : extraction du token depuis l'en-tête Authorization, validation de la signature RS256, vérification de l'expiration, consultation de la blacklist Redis, et injection du contexte de sécurité.

Lecture du diagramme

La validation du token se fait en deux etapes : (1) le `BearerTokenFilter` extrait le token de l'en-tete `HTTP Authorization: Bearer <token>` via une expression reguliere; (2) le `RemoteAuthAdapter` appelle le service Auth externe pour valider le token et recuperer les claims utilisateur. L'utilisateur local est cree ou mis a jour dans `UserLocalRepository` apres validation, permettant la tracabilite des actions dans les logs d'audit. Les permissions sont cached dans Redis avec une cle `user:permissions:<userId>` et un TTL de 1 heure, evitant des appels reseau repetitifs. En cas d'echec de validation (token expire, signature invalide, utilisateur revoke), une exception `InvalidTokenException` est levee et mappee en HTTP 401 Unauthorized par le `GlobalExceptionHandler`.

5.1.4 Mechanisme de rafraichissement de token

FleetMan implemente un mecanisme de refresh token pour les sessions longues :

- **Access token** : Duree de vie courte (15 minutes), contient toutes les permissions, utilise pour chaque requete API.
- **Refresh token** : Duree de vie longue (7 jours), stocke en HTTP-only cookie secure, utilise uniquement pour obtenir un nouvel access token.
- **Rotation** : A chaque utilisation du refresh token, un nouveau refresh token est genere et l'ancien est revoke (rotation pour limiter le risque de vol).

Cette strategie equilibre securite (access token court limite l'exposition en cas de vol) et experience utilisateur (pas de re-login toutes les 15 minutes).

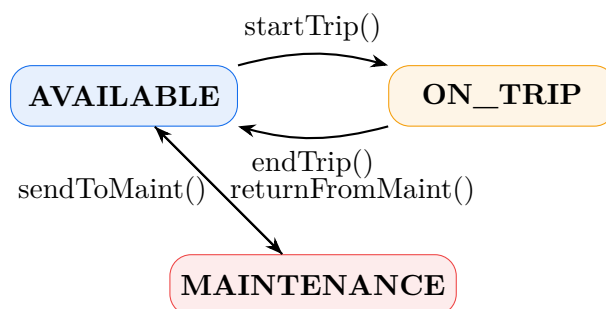
5.2 Module Vehicule — Machine a etats

Fig. 5.2 : Machine a etats du vehicule FleetMan. Le statut AVAILABLE est l'etat initial. Seul un vehicule AVAILABLE peut demarrer un trajet. Un vehicule en MAINTENANCE ne peut pas etre affecte.

5.3 Module Trajet — Diagramme d'états

La figure 5.3 presente le cycle de vie complet d'un trajet depuis sa creation jusqu'a sa cloture.

5.3.1 Etats du trajet et transitions

Le cycle de vie d'un trajet comprend 5 etats principaux :

1. **PLANNED** : Etat initial apres creation. Le trajet est planifie mais pas encore demarre. Le gestionnaire peut modifier les details (depart, arrivee, conducteur, vehicule).
2. **READY** : Le conducteur a valide sa disponibilite et le vehicule est pret (statut AVAILABLE, niveau carburant suffisant, aucun document expire). Transition automatique depuis PLANNED ou declenchee manuellement.
3. **ONGOING** : Trajet en cours d'execution. Le conducteur envoie la telemetrie GPS toutes les 30 secondes. Le georeperage est actif. Les evenements de conduite (exces de vitesse, freinages brusques) sont detectes en temps reel.
4. **COMPLETED** : Trajet termine avec succes. La distance reelle, la duree et la consommation carburant sont automatiquement calculees. Le score de conduite du conducteur est mis a jour.
5. **CANCELLED** : Trajet annule avant ou pendant l'execution. Necessite un motif obligatoire (panne vehicule, indisponibilite conducteur, mission annulee par le client).

5.3.2 Regles metier des transitions

Chaque transition est soumise a des preconditions strictes :

- **PLANNED** → **READY** : Vehicule AVAILABLE, conducteur ACTIVE, pas de conflit d'affectation, documents vehicule et conducteur valides.
- **READY** → **ONGOING** : Declenchee uniquement par le conducteur via l'application mobile ou le dashboard. Horodatage GPS initial obligatoire.
- **ONGOING** → **COMPLETED** : Distance minimale de 1 km parcourue, duree minimale de 5 minutes. Horodatage GPS final obligatoire.
- * → **CANCELLED** : Possible depuis PLANNED, READY ou ONGOING.

Depuis ONGOING, necessite validation du gestionnaire (annulation tardive).

5.3.3 Calculs automatisés lors de la completion

Au passage en etat COMPLETED, plusieurs calculs sont executes automatiquement :

1. **Distance reelle** : Sommation des distances entre points GPS consecutifs (formule de Haversine).
2. **Duree reelle** : Difference entre `endedAt` et `startedAt`, decomposee en heures effectives (hors pauses).
3. **Consommation estimee** : Calcul base sur la consommation moyenne du vehicule (L/100km) et la distance reelle.
4. **Cout estime** : Distance $\times$ cout kilométrique (carburant + amortissement + maintenance repartie).
5. **Score de conduite** : Agregation des evenements detectes (exces vitesse, freinages brusques, accelerations, virages serres) via l'algorithme de scoring (voir algorithme ??).
6. **Ecart par rapport au plan** : Comparaison distance reelle vs estimee, duree reelle vs estimee, avec calcul des taux d'ecart en pourcentage.

Lecture du diagramme

L'etat ONGOING est l'etat actif durant lequel le conducteur envoie des donnees de telemetrie GPS toutes les 30 secondes. C'est dans cet etat que le georeperage est actif et que les evenements de conduite sont detectes en temps reel. L'etat COMPLETED calcule automatiquement la distance parcourue (somme des segments GPS via formule de Haversine), la duree du trajet (difference timestamps debut/fin), la consommation carburant estimee (distance $\times$ conso moyenne vehicule), et le score de conduite du conducteur (penalites basees sur les evenements detectes). Ces calculs sont executes de facon asynchrone dans un job reactif pour ne pas bloquer la reponse HTTP au conducteur. L'etat CANCELLED preserve toutes les donnees collectees jusqu'au point d'annulation pour analyse ulterieure (telemetrie partielle, evenements detectes, carburant consomme).

5.3.4 Persistance de la telemetrie GPS

La telemetrie GPS suit une strategie de stockage hybride :

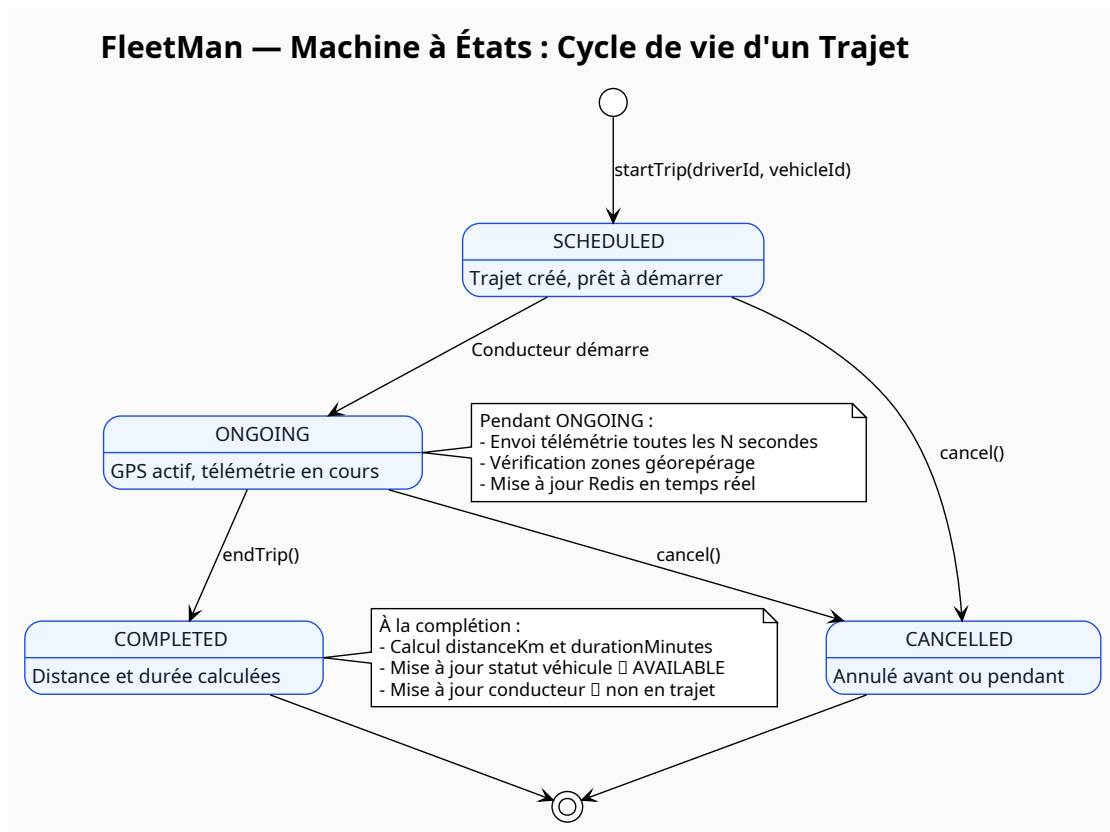


Fig. 5.3 : Diagramme d'états : Cycle de vie d'un Trajet. Les transitions sont déclenchées par les actions du conducteur (`startTrip`, `endTrip`) ou du gestionnaire (`cancel`). Les états `PLANNED` et `READY` sont des états préparatoires sans télémétrie GPS. L'état `ONGOING` est l'état actif durant lequel le conducteur envoie des données de télémétrie toutes les 30 secondes et le géorepérage est active. L'état `COMPLETED` est terminal et déclenche les calculs automatisés (distance, durée, consommation, score de conduite). L'état `CANCELLED` peut être atteint depuis n'importe quel état non terminal et nécessite un motif obligatoire.

- **Redis (temps reel)** : Derniere position de chaque vehicule active, avec TTL de 5 minutes. Utilisee pour l’affichage carte temps reel sur le dashboard.
- **Kafka (streaming)** : Tous les points GPS publies sur topic `fleetman.telemetry.gps` pour consommation par services externes (georeperage, analytics, replay).
- **PostgreSQL (historique)** : Points GPS decimes (1 point toutes les 2 minutes) stockes dans la table `trip_telemetry` pour les replays et analyses historiques. Partition mensuelle pour optimiser les requetes.

Cette architecture garantit des performances optimales pour l’affichage temps reel (Redis) tout en preservant l’historique complet (Kafka + PostgreSQL).

5.4 Module Georeperage — Diagramme de sequence

La figure 5.4 illustre le flux de telemetrie GPS et le declenchement automatique des alertes de georeperage.

5.4.1 Architecture du module de georeperage

Le georeperage FleetMan s’appuie sur l’extension PostGIS de PostgreSQL pour les calculs geometriques :

1. **Definition des zones** : Le gestionnaire trace des polygones sur une carte interactive (React Leaflet). Les coordonnees sont envoyees au backend et persistees en format GeoJSON dans la colonne `boundary` `GEOMETRY(POLYGON, 4326)`.
2. **Types de zones** : `INCLUSION` (le vehicule doit rester a l’interieur), `EXCLUSION` (le vehicule ne doit pas entrer), `CHECKPOINT` (le vehicule doit passer par la zone).
3. **Detection en temps reel** : A chaque reception de telemetrie GPS, une requete PostGIS verifie si le point est dans/hors des zones actives via la fonction `ST_Contains`.
4. **Alertes** : Violation detectee → evenement Kafka → notification push au gestionnaire (mobile) et email (bureau).

5.4.2 Requete PostGIS de detection

La detection de violation utilise la requete spatiale suivante :

```
SELECT g.id, g.name, g.fence_type
```

```
FROM geofences g
WHERE g.tenant_id = :tenantId
      AND g.status = 'ACTIVE'
      AND (
        (g.fence_type = 'INCLUSION' AND NOT ST_Contains(g.boundary, :point))
        OR
        (g.fence_type = 'EXCLUSION' AND ST_Contains(g.boundary, :point))
      );
```

L'index spatial `CREATE INDEX idx_geofence_boundary ON geofences USING GIST(boundary)` garantit une complexité en $O(\log n)$ même avec des milliers de zones.

5.4.3 Algorithme de dedoublonnage des alertes

Pour éviter le spam d'alertes (véhicule oscillant à la frontière d'une zone), un mécanisme de dedoublonnage est implémenté :

1. Stocker dans Redis la clé `alert:<vehicleId>:<geofenceId>` avec TTL de 5 minutes.
2. Avant d'envoyer une alerte, vérifier l'existence de cette clé.
3. Si la clé existe, ignorer l'alerte (déjà envoyée récemment).
4. Si la clé n'existe pas, envoyer l'alerte et créer la clé.

Ce mécanisme limite à 1 alerte par zone toutes les 5 minutes par véhicule.

FleetMan — Diagramme de Séquence : Envoi de télémétrie et alerte géorepérage

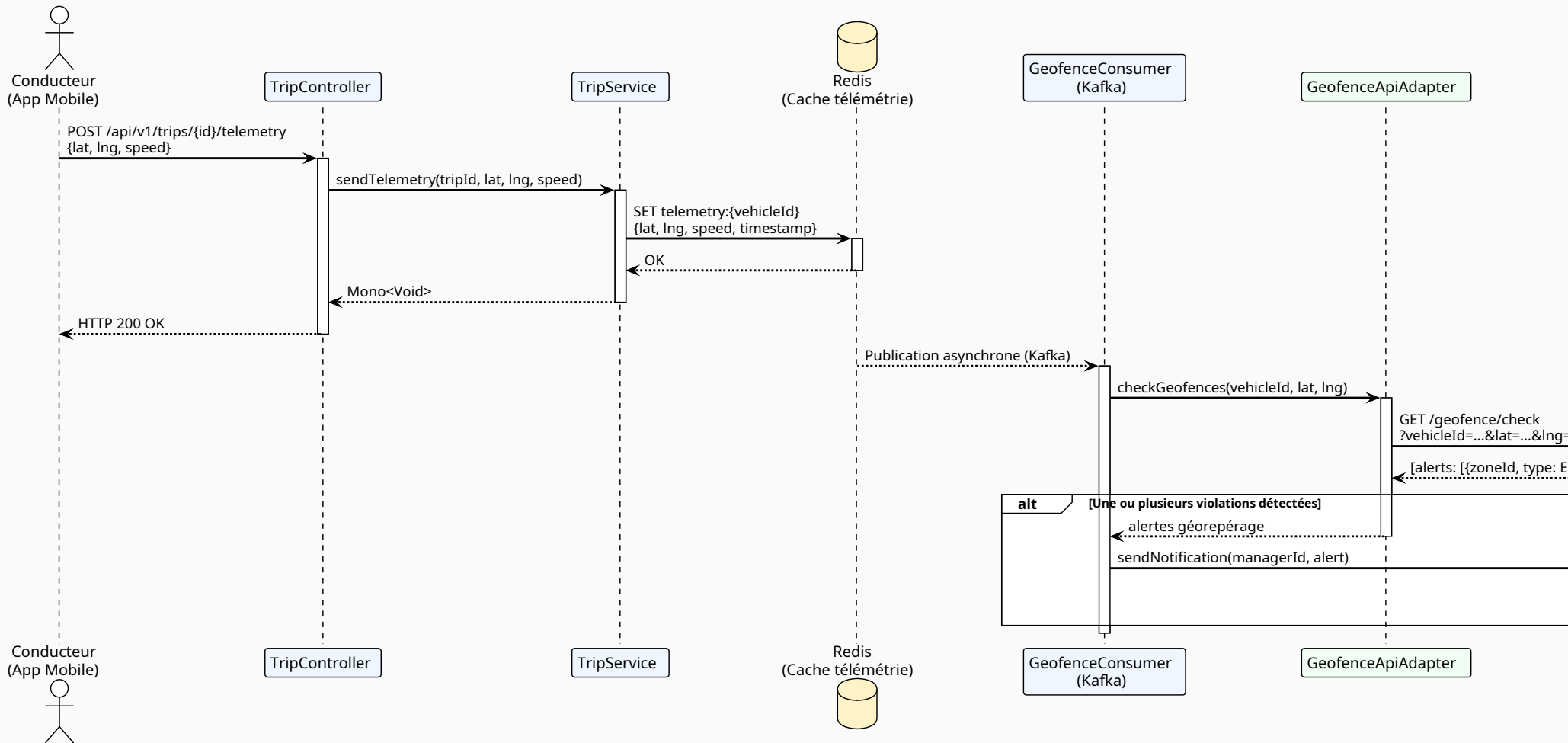


Fig. 5.4 : Diagramme de sequence : Envoi de telemetrie et alerte de georeperage. Les fleches pleines representent des appels synchrones reactifs (Mono/Flux) ; les fleches tiretees representent des messages asynchrones Kafka en mode fire-and-forget. Apres reception de la telemetrie (1), l'application valide le format GPS et l'appartenance du vehicule au tenant ; stocke les coordonnees GPS dans Redis (2) avec cle `vehicle:position:<vehicleId>` et TTL de 5 minutes pour un acces temps reel ; persiste le point decime en PostgreSQL si intervalle ≥ 2 minutes depuis dernier stockage (3) ; publie l'evenement **TelemetryReceived** sur Kafka (4). Le consommateur geofence (service reactif autonome) recoit l'evenement, execute la requete PostGIS de detection de violations (5), consulte le cache de dedoublonnage Redis (6), et envoie une notification push via l'adapter externe (7) si violation detectee et non dedoublonnee. Ce flux asynchrone garantit que l'envoi de telemetrie par le conducteur retourne immediatement (latence $< 50\text{ms}$) sans etre bloque par les traitements lourds (detection geofence, notifications).

Lecture du diagramme

Après reception de la telemetrie (1), l'application stocke les coordonnees GPS dans Redis (2) pour un acces temps reel avec cle `vehicle:position:<vehicleId>` et TTL de 5 minutes. L'evenement est ensuite publie sur Kafka (3) sur le topic `fleetman.telemetry.gps`. Le consommateur geofence (service reactif autonome s'executant dans un thread pool dedie) recoit l'evenement de facon asynchrone, verifie les zones actives via requete PostGIS (4), consulte le cache Redis de dedoublonnage (5), et envoie une notification push (6) si une violation est detectee et non dedoublonnee (derniere alerte pour cette zone > 5 minutes). Ce flux asynchrone garantit que l'envoi de telemetrie par le conducteur n'est pas bloque par le traitement du georeperage (latence P95 < 50ms pour la reponse HTTP au conducteur). Le consommateur geofence traite les evenements par batch de 100 pour optimiser les performances, avec un flush automatique toutes les 5 secondes. En cas d'erreur de notification (service externe indisponible), l'evenement est republie sur un topic DLQ (Dead Letter Queue) pour retraitement manuel.

5.4.4 Performances et optimisations

Les optimisations suivantes garantissent des performances temps reel :

- **Index spatial GIST** : Index sur la colonne `boundary` pour recherches spatiales en $O(\log n)$.
- **Index partiel sur statut** : `WHERE status='ACTIVE'` pour ignorer les zones archivees.
- **Cache Redis des zones actives** : Les zones actives sont cached 15 minutes pour eviter des requetes PostgreSQL repetitives.
- **Traitement batch** : Le consommateur Kafka traite les evenements par batch de 100 pour reduire les requetes SQL.
- **Partition Kafka** : Le topic `telemetry` est partitionne par `vehicleId` pour garantir l'ordre des points GPS d'un meme vehicule.

Les tests de performance montrent une capacite de traitement de **10 000 points GPS/seconde** avec latence P95 < 200ms pour la detection et l'envoi d'alerte.

Chapitre 6

Module Operations Terrain

6.1 Machine a etats de l'Incident

La figure 6.1 presente la machine a etats de l'entite Incident, l'une des plus complexes du module operations.

6.1.1 Cycle de vie complet d'un incident

Un incident traverse 4 etats principaux avec des transitions controlees :

1. **OPEN** : Etat initial apres declaration par le conducteur. L'incident contient une description, une severite (LOW, MEDIUM, HIGH, CRITICAL), une localisation GPS optionnelle, et des photos optionnelles. Le gestionnaire recoit une notification immediate si la severite est HIGH ou CRITICAL.
2. **IN_PROGRESS** : Un technicien ou le gestionnaire a pris en charge l'incident. Cet etat autorise la mise a jour du diagnostic, l'ajout de commentaires, et le changement de severite si necessaire (escalade/descalade).
3. **RESOLVED** : L'incident est resolu techniquement (reparation effectuee, probleme corrige). Les champs **resolvedAt**, **resolvedBy**, et **resolutionNotes** sont obligatoires. Le vehicule peut reprendre le service si aucun autre incident CRITICAL n'est actif.
4. **CLOSED** : Etat terminal apres verification par le gestionnaire. L'incident est archive et ne peut plus etre modifie. Le cout final (pieces + main d'oeuvre) est obligatoire pour les incidents de severite HIGH et CRITICAL.

6.1.2 Regles metier et invariants

L'entite **Incident** encapsule plusieurs regles metier :

- **Blocage vehicule** : Un incident CRITICAL bloque automatiquement le vehicule (statut → MAINTENANCE). Le vehicule ne peut pas demarrer

de trajet tant qu'un incident CRITICAL est actif.

- **Escalade automatique** : Si un incident MEDIUM reste en etat OPEN plus de 48h, il est automatiquement escalade en HIGH (job nocturne).
- **Resolution obligatoire avant cloture** : La transition OPEN ou IN_PROGRESS → CLOSED est interdite. Il faut passer par RESOLVED.
- **Horodatage automatique** : Les champs `openedAt`, `assignedAt`, `resolvedAt`, `closedAt` sont renseignes automatiquement lors des transitions.
- **Cost obligatoire** : Pour les incidents HIGH et CRITICAL, le champ `cost` est obligatoire en etat RESOLVED ou CLOSED.

6.1.3 Methodes metier de l'entite Incident

L'entite expose 5 methodes metier encapsulant la logique de transition :

```
public class Incident {
    public void assign(UUID technicianId) {
        if (this.status != OPEN && this.status != IN_PROGRESS) {
            throw new InvalidStateException("Cannot assign a resolved incident");
        }
        this.status = IN_PROGRESS;
        this.assignedTo = technicianId;
        this.assignedAt = LocalDateTime.now();
    }

    public void resolve(String resolutionNotes, BigDecimal cost) {
        if (this.status == CLOSED) {
            throw new InvalidStateException("Cannot resolve a closed incident");
        }
        if ((this.severity == HIGH || this.severity == CRITICAL) && cost == null) {
            throw new ValidationException("Cost is required for high/critical incidents")
        }
        this.status = RESOLVED;
        this.resolvedAt = LocalDateTime.now();
        this.resolutionNotes = resolutionNotes;
        this.cost = cost;
    }

    public void close() { /* ... */ }
```

```
public void escalate(IncidentSeverity newSeverity) { /* ... */ }  
public boolean isCritical() { return severity == CRITICAL || severity == HIGH; }  
}
```

Lecture du diagramme

Les methodes metier `resolve()`, `close()` et `updateStatus()` sont definies dans l'entite domaine `Incident`, pas dans le service. Cette encapsulation garantit que les invariants metier sont toujours respectes, independamment du chemin d'accès (REST API, job batch, integration Kafka). La methode `isCritical()` retourne `true` si la severite est `HIGH` ou `CRITICAL`, declenchant une alerte prioritaire au gestionnaire (notification push + SMS) et un blocage automatique du vehicule. Ce blocage est implemente par une mise a jour du statut vehicule en `MAINTENANCE` dans le meme contexte transactionnel que la creation de l'incident (atomicite garantie). L'etat `RESOLVED` necessite des notes de resolution obligatoires pour la tracabilite et le retour d'experience (knowledge base). Ces notes sont indexees en full-text dans PostgreSQL pour recherche ulterieure (ex : "panne electrique", "fuite d'huile"). Les transitions illegales (ex : `CLOSED` → `OPEN`) levent une exception `InvalidStateException` mappee en HTTP 409 Conflict par le global exception handler.

6.1.4 Statistiques et KPIs des incidents

Le module incidents expose plusieurs indicateurs cles :

- **MTTR (Mean Time To Resolve)** : Duree moyenne entre `openedAt` et `resolvedAt`, calculee par severite.
- **MTBF (Mean Time Between Failures)** : Duree moyenne entre deux incidents d'un meme vehicule, indicateur de fiabilite.
- **Taux d'incidents critiques** : % d'incidents `HIGH/CRITICAL` par rapport au total, cible < 10%.
- **Cout moyen par incident** : Agregation des couts de resolution, decompose par type d'incident (panne mecanique, accident, vandalisme).
- **Top vehicules problematiques** : Classement des vehicules par nombre d'incidents sur les 30 derniers jours.

Ces KPIs sont calcules quotidiennement par un job planifie et exposes via l'endpoint `GET /api/v1/operations/incidents/stats`.

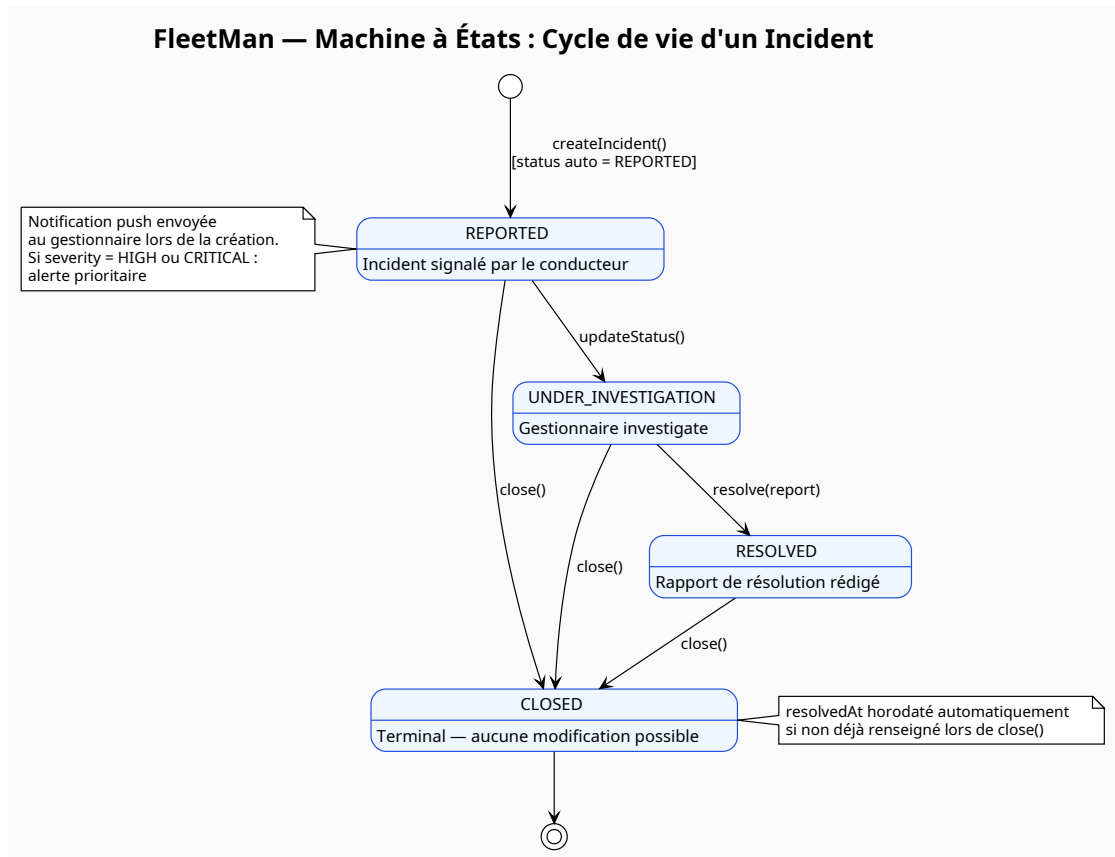


Fig. 6.1 : Diagramme d'états : Cycle de vie d'un Incident. L'état CLOSED est terminal : aucune modification n'est possible après clôture (immutabilité pour l'audit). La transition vers RESOLVED horodate automatiquement le champ `resolvedAt` et nécessite des notes de résolution obligatoires. La méthode métier `isCritical()` retourne `true` si la sévérité est HIGH ou CRITICAL, déclenchant une notification prioritaire au gestionnaire et un blocage automatique du véhicule. L'état IN_PROGRESS permet les mises à jour itératives (ajout de photos, commentaires, pièces commandées) sans changer d'état. La transition RESOLVED → IN_PROGRESS est autorisée en cas de régression (incident résolu mais problème réapparu).

6.2 Diagramme de sequence — Declaration d'un incident

La figure 6.2 presente le flux complet de creation d'un incident depuis la declaration par le conducteur.

6.2.1 Flux de declaration d'incident

La declaration d'un incident suit un processus en 7 etapes :

1. **Soumission formulaire** : Le conducteur remplit le formulaire sur l'application mobile (description, severite estimee, localisation GPS automatique, photos optionnelles jusqu'a 5 fichiers).
2. **Upload photos** : Les photos sont uploadées en parallele sur S3 via presigned URLs. Les URLs finales sont incluses dans la requete de creation.
3. **Validation vehicule** : Le service verifie l'existence du vehicule et son appartenance au tenant du conducteur.
4. **Validation conducteur** : Verification que le conducteur est bien affecte au vehicule au moment de l'incident (consultation table `assignments`).
5. **Construction entite** : Creation de l'entite `Incident` avec validation des invariants dans le constructeur (severite non nulle, description minimale 10 caracteres).
6. **Persistence** : Sauvegarde dans PostgreSQL avec generation d'un code incident unique (format INC-YYYYMMDD-XXXX).
7. **Publication evenement** : Evenement `IncidentCreated` publie sur Kafka pour notifications asynchrones et audit.

6.2.2 Regles de validation

La creation d'incident applique plusieurs validations :

- **Description** : Longueur entre 10 et 1000 caracteres.
- **Severite** : Enum valide (LOW, MEDIUM, HIGH, CRITICAL).
- **Localisation GPS** : Coordonnees valides si fournies ($-90 \leq \text{lat} \leq 90$, $-180 \leq \text{lon} \leq 180$).

- **Photos** : Maximum 5 fichiers, formats autorises (JPEG, PNG), taille max 5 MB par fichier.
- **Vehicule** : Doit exister et appartenir au tenant.
- **Conducteur** : Doit etre affecte au vehicule au moment de la declaration.

En cas d'echec de validation, une exception `ValidationException` est levee avec details des erreurs, mappee en HTTP 400 Bad Request.

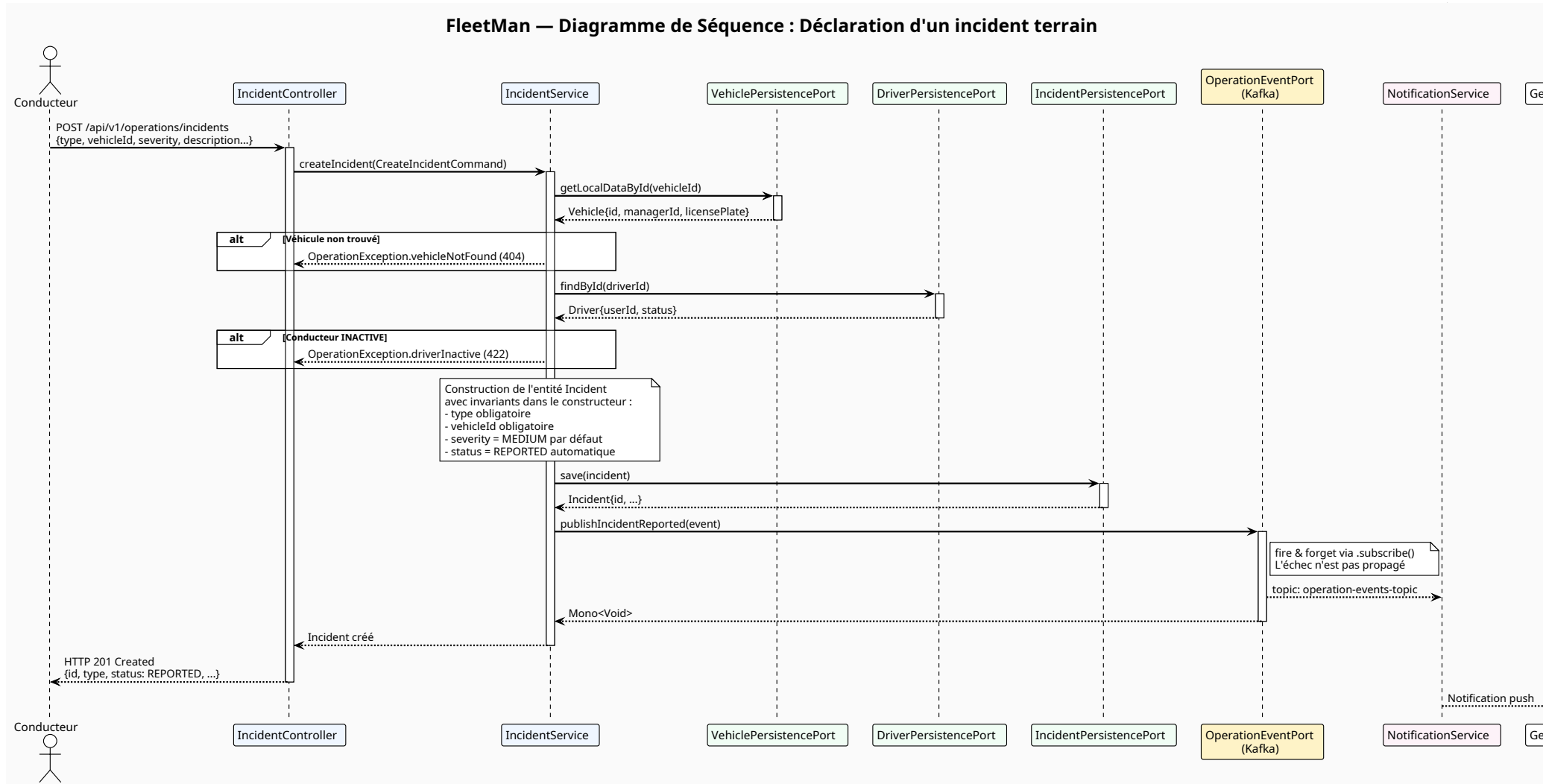


Fig. 6.2 : Diagramme de séquence : Declaration d'un incident terrain. Le service vérifie l'existence du véhicule et du conducteur avant de persister l'incident et de publier l'événement Kafka. La création d'un incident suit un flux en 7 étapes : (1) Vérification du véhicule via **VehiclePersistencePort**, avec consultation du cache Redis en premier pour limiter les requêtes PostgreSQL ; (2) Vérification de l'affectation conducteur-véhicule via requête SQL sur **assignments** avec filtre sur statut **IN_PROGRESS** et chevauchement temporel ; (3) Construction de l'entité **Incident** avec validation des invariants dans le constructeur compact (record) ou explicite (classe) ; (4) Persistance en base de données via **IncidentPersistencePort** implémentée par **IncidentR2dbcAdapter** ; (5) Si severite **HIGH** ou **CRITICAL**, mise à jour atomique du statut véhicule en **MAINTENANCE** (même transaction) ; (6) Publication d'un événement **IncidentCreated** sur Kafka en mode fire-and-forget avec gestion d'erreur silencieuse ; (7) Retour de l'entité persistée au client avec **HTTP 201 Created** et header **Location** contenant l'URI de la ressource. Ce dernier point garantit que l'échec de la

Lecture du diagramme

La creation d'un incident suit un flux en 5 etapes : (1) Verification du vehicule via `VehiclePersistencePort`, avec consultation du cache Redis en premier pour limiter les requetes PostgreSQL ; (2) Verification optionnelle du conducteur via requete sur la table `assignments` pour s'assurer qu'il est bien affecte au vehicule ; (3) Construction de l'entite avec validation des invariants du constructeur (description non vide, severite valide, coordonnees GPS dans les bornes) ; (4) Persistence en base de donnees via l'adapter R2DBC avec generation automatique du code incident (format INC-YYYYMMDD-XXXX) ; (5) Publication d'un evenement Kafka `IncidentCreated` en mode fire-and-forget (opérateur `.subscribe()` sans gestion d'erreur bloquante). Ce dernier point garantit que l'echec de la notification ne fait pas echouer la creation de l'incident (principe de resilience). Si la severite est HIGH ou CRITICAL, une mise a jour atomique du statut vehicule en MAINTENANCE est effectuee dans la meme transaction (garantie de coherence). L'evenement Kafka contient toutes les donnees necessaires pour les consommateurs (notification service, audit service, analytics) : ID incident, code, severite, vehicule, conducteur, localisation, timestamp. Le consommateur notification envoie un SMS au gestionnaire si severite CRITICAL, un email si HIGH, et une notification push dans tous les cas.

6.2.3 Gestion des photos d'incident

Les photos d'incident suivent un flux S3 optimise :

1. Le frontend demande des presigned URLs au backend via `POST /api/v1/operations/inc` avec les metadonnees (noms fichiers, types MIME, tailles).
2. Le backend genere des presigned URLs S3 avec expiration de 15 minutes et les retourne au frontend.
3. Le frontend uploades les photos directement sur S3 via les presigned URLs (upload parallele, pas de transit par le backend).
4. Le frontend soumet le formulaire incident avec les URLs S3 finales des photos uploadées.
5. Le backend valide l'existence des objets S3 avant de persister l'incident.

Cette architecture offload l'upload sur S3 et reduit la charge sur le backend.

Chapitre 7

Module Planification et Affectations

7.1 Machine a etats du Planning

La figure 7.1 presente le cycle de vie d'un planning de service.

7.1.1 Etats et transitions du planning

Le planning de service traverse 4 etats avec des regles de transition strictes :

- (a) **DRAFT** : Etat initial apres creation. Le gestionnaire peut librement ajouter, modifier, supprimer des affectations. Les conducteurs ne voient pas le planning. Aucune validation de conflits n'est appliquee (permet l'edition rapide).
- (b) **PUBLISHED** : Le gestionnaire a valide et publie le planning. Les affectations sont visibles aux conducteurs dans leur application mobile. Toutes les affectations ont ete validees (pas de conflits vehicule/conducteur). Des modifications sont encore possibles mais necessitent une nouvelle publication (transition vers DRAFT puis re-publication).
- (c) **ACTIVE** : Le planning est en cours d'execution (date de debut atteinte). Les conducteurs executent leurs affectations. Les modifications sont limitees (ajout d'affectations futures autorise, suppression d'affectations passees interdite).
- (d) **COMPLETED** : Toutes les affectations du planning sont terminees ou annulees. Etat terminal en lecture seule, utilise pour l'archivage et les analyses historiques.

7.1.2 Regles de modification selon l'état

Les modifications autorisees dependent de l'état du planning :

Tab. 7.1 : Matrice des operations autorisees par etat de planning

Operation	DRAFT	PUBLISHED	ACTIVE	COMPLETED
Ajouter affectation	Oui	Vers DRAFT	Futur uniquement	Non
Modifier affectation	Oui	Vers DRAFT	Futur uniquement	Non
Supprimer affectation	Oui	Vers DRAFT	Non	Non
Changer periode	Oui	Non	Non	Non
Publier	Oui	Deja publie	Non	Non
Archiver	Oui	Oui	Non	Oui

7.1.3 Validation avant publication

Avant la transition DRAFT $\rightarrow$ PUBLISHED, une validation complete est executee :

- (a) **Couverture temporelle** : Toutes les heures de service definies doivent avoir au moins une affectation (pas de trou dans le planning).
- (b) **Detection conflits vehicules** : Verification qu'aucun vehicule n'a deux affectations simultanees.
- (c) **Detection conflits conducteurs** : Verification qu'aucun conducteur n'a deux affectations simultanees.
- (d) **Validation disponibilites** : Tous les conducteurs affectes doivent etre en statut ACTIVE (pas ON_LEAVE, pas INACTIVE).
- (e) **Validation documents** : Tous les documents legaux (permis conducteurs, assurances vehicules) doivent etre valides sur la periode du planning.
- (f) **Validation capacites** : Pour les missions avec passagers/fret, verification que les capacites vehicules sont respectees.

Si une seule validation echoue, la publication est refusee avec un rapport d'erreurs detaille (code, message, affectations concernees).

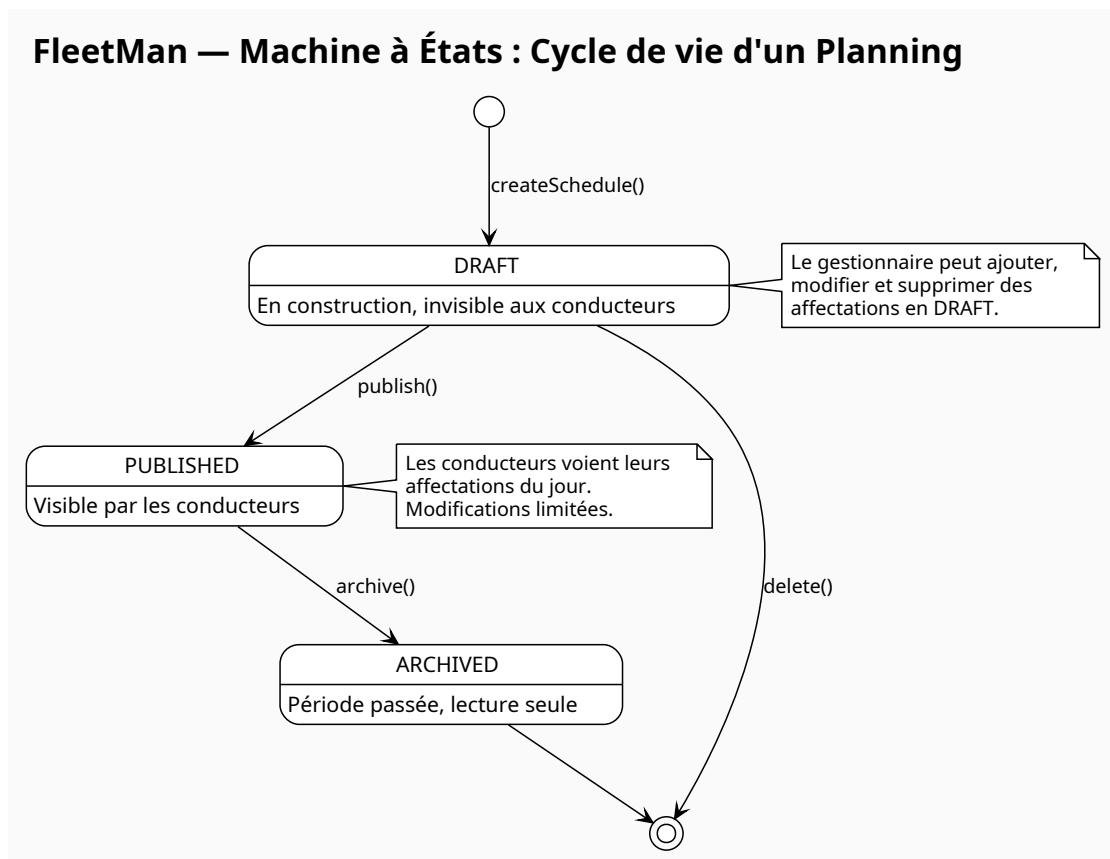


Fig. 7.1 : Diagramme d'états : Cycle de vie d'un Planning. En état DRAFT, le gestionnaire peut modifier librement les affectations sans validation de conflits (édition rapide). La publication déclenche une validation complète (conflits, disponibilités, documents) et rend le planning visible aux conducteurs. L'état ACTIVE est atteint automatiquement lorsque la date de début du planning est atteinte (transition déclenchée par job planifié quotidien). L'état COMPLETED est également atteint automatiquement lorsque toutes les affectations sont terminées (statut COMPLETED ou CANCELLED). La transition PUBLISHED → DRAFT permet au gestionnaire de dépublier le planning pour effectuer des modifications importantes (les conducteurs ne voient plus le planning pendant l'édition). La transition ACTIVE → COMPLETED nécessite que 100% des affectations soient dans un état terminal (COMPLETED ou CANCELLED).

7.2 Detection de conflits — Diagramme de sequence

La figure 7.2 illustre l’algorithme de detection de conflits d’affectation, fonctionnalite critique du module de planification.

7.2.1 Types de conflits detectes

Le systeme detecte 4 types de conflits :

- (a) **Conflit vehicule** : Le vehicule est deja affecte a un autre conducteur sur une plage horaire chevauchante.
- (b) **Conflit conducteur** : Le conducteur est deja affecte a un autre vehicule sur une plage horaire chevauchante.
- (c) **Conflit maintenance** : Le vehicule est planifie en maintenance sur une periode chevauchante.
- (d) **Conflit disponibilite** : Le conducteur a declare une indisponibilite (conge, formation, maladie) sur une periode chevauchante.

7.2.2 Algorithme de detection optimise

La detection utilise des requetes SQL optimisees avec index partiels :

```
-- Detection conflits vehicule
SELECT COUNT(*) FROM assignments
WHERE vehicle_id = :vehicleId
  AND tenant_id = :tenantId
  AND status IN ('PENDING', 'IN_PROGRESS')
  AND (
    (start_datetime <= :startDatetime AND end_datetime > :startDatetime)
    OR (start_datetime < :endDatetime AND end_datetime >= :endDatetime)
    OR (start_datetime >= :startDatetime AND end_datetime <= :endDatetime)
  );

-- Index partiel pour performances
CREATE INDEX idx_assignments_active_vehicle
ON assignments(vehicle_id, start_datetime, end_datetime)
WHERE status IN ('PENDING', 'IN_PROGRESS');
```

Les index partiels sur le statut garantissent une complexite en $O(\log n)$ meme avec des milliers d'affectations (seules les actives sont indexees).

7.2.3 Gestion des chevauchements temporels

Le chevauchement de deux plages $[s_1, e_1]$ et $[s_2, e_2]$ est detecte par la condition :

$$\text{overlap} = (s_1 \leq s_2 < e_1) \vee (s_1 < e_2 \leq e_1) \vee (s_2 \leq s_1 < e_2) \vee (s_2 < e_1 \leq e_2) \quad (7.1)$$

Cette formule peut etre simplifiee en :

$$\text{overlap} = \max(s_1, s_2) < \min(e_1, e_2) \quad (7.2)$$

Cependant, la version SQL explicite est preferee pour la clarte et l'utilisation optimale des index B-tree sur les timestamps.

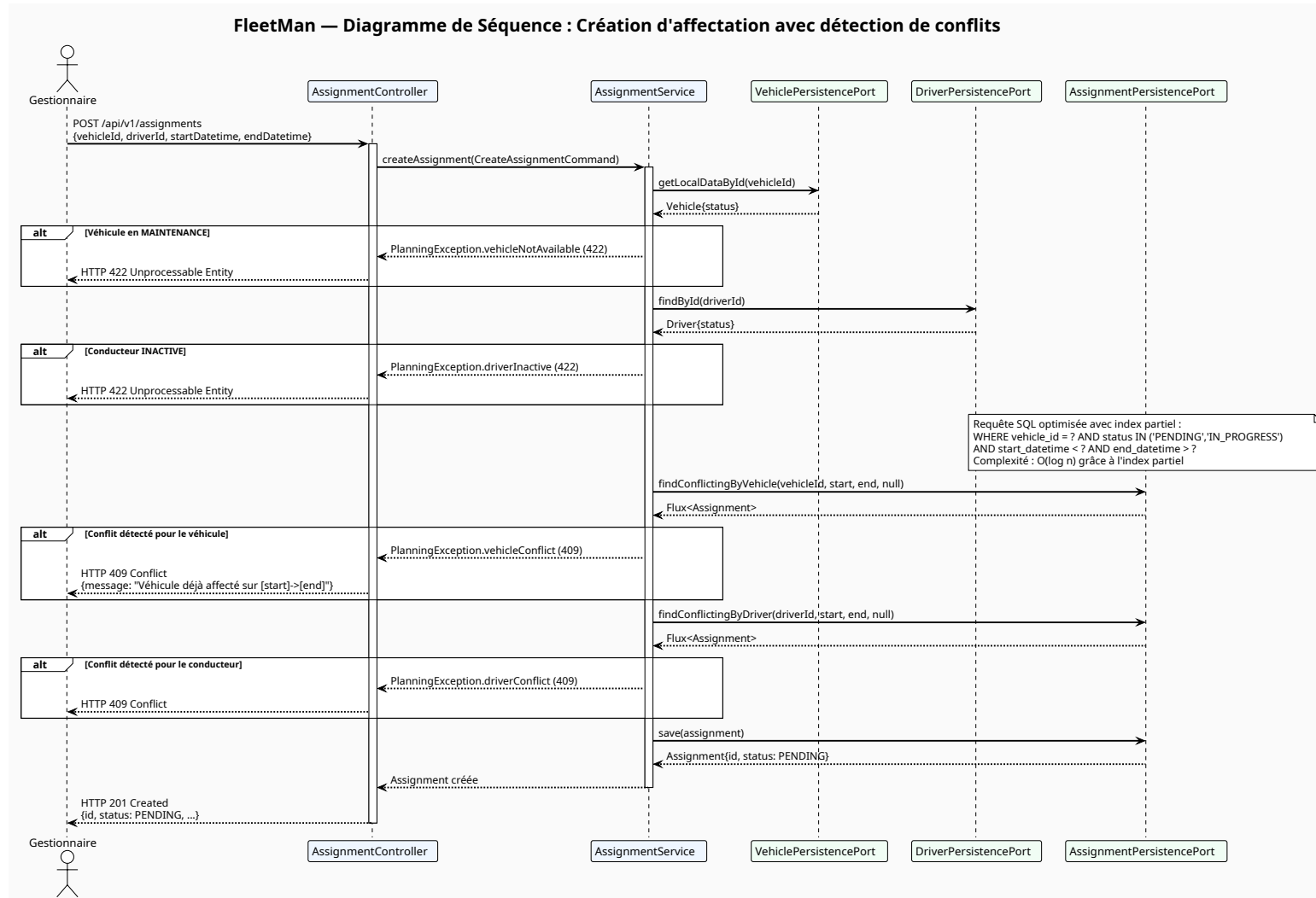


Fig. 7.2 : Diagramme de séquence : Création d'affectation avec détection de conflits. Les deux requêtes SQL de détection utilisent des index partiels sur les statuts actifs (PENDING, IN_PROGRESS), garantissant une complexité en $O(\log n)$. Le flux complet comprend 9 étapes : (1) Réception de la requête HTTP POST avec payload JSON contenant vehicleId, driverId, startDatetime, endDatetime ; (2) Validation du véhicule : consultation cache Redis puis fallback PostgreSQL si cache miss ; (3) Vérification du statut véhicule (doit être AVAILABLE, pas MAINTENANCE ni ON_TRIP) ; (4) Validation du conducteur : existence et statut ACTIVE ; (5) Détection conflits véhicule via requête SQL avec index partiel sur (vehicle_id, start_datetime, end_datetime) WHERE status IN ('PENDING', 'IN_PROGRESS') ; (6) Détection conflits conducteur via requête similaire sur driver_id ; (7) Vérification maintenances planifiées du véhicule sur la période ; (8) Vérification indisponibilités déclarées du conducteur ; (9) Si aucun conflit, persistance de l'affectation avec statut PENDING et publication événement Kafka pour notification au conducteur. En cas de conflit a

Lecture du diagramme

La detection de conflits s'effectue en deux etapes sequentielles :

(1) Verification du vehicule : requete SQL sur les affectations actives (PENDING ou IN_PROGRESS) chevauchant la plage horaire demandee, utilisant un index partiel `CREATE INDEX idx_assignments_active_vehicle ON assignments(vehicle_id, start_datetime, end_datetime) WHERE status IN ('PENDING', 'IN_PROGRESS')` pour un acces en $O(\log n)$; (2) Verification du conducteur : meme logique avec index similaire sur `driver_id`. Les index partiels PostgreSQL (condition WHERE dans la definition de l'index) garantissent une performance optimale meme avec des milliers d'affectations, car seules les affectations actives (PENDING, IN_PROGRESS) sont indexees, excluant les affectations terminees (COMPLETED, CANCELLED) qui representent la majorite du volume apres quelques mois d'utilisation. En cas de conflit, une exception `PlanningException` est levee avec le code PLN-006 (conflit vehicule) ou PLN-007 (conflit conducteur), mappee en HTTP 409 Conflict par le global exception handler avec un payload JSON contenant les details du conflit (ID affectation en conflit, plage horaire, vehicule/conducteur concerne). Le frontend peut ainsi afficher un message d'erreur precis avec proposition de creneaux alternatifs. Les verifications de maintenance et d'indisponibilite utilisent des requetes similaires sur les tables `maintenances` et `driver_availabilities` avec index temporels equivalents.

7.2.4 Optimisations de performance

Plusieurs optimisations garantissent des temps de reponse rapides :

- **Cache Redis des conflits** : Les conflits recemment detectes sont cached 5 minutes avec cle `conflict:<vehicleId>:<timestamp>`. Evite de reexecuter la meme requete pour des tentatives repetees.
- **Requetes paralleles** : Les 4 verifications (conflit vehicule, conflit conducteur, maintenance, disponibilite) sont executees en parallele via `Mono.zip()` pour reduire la latence totale.
- **Pagination des resultats** : Si un planning contient des milliers d'affectations, la validation est paginee par batch de 100 pour eviter l'OutOfMemory.

- **Index composites** : Index sur (tenant_id, vehicle_id, start_datetime) pour exploiter la selectivite maximale (tenant d'abord, puis vehicule, puis date).

Les tests de performance montrent une latence P95 < 150ms pour la creation d'affectation avec detection de conflits sur un planning de 1000 affectations.

7.3 Algorithme de detection de conflits

Algorithme 1 Detection de conflits lors d'une creation d'affectation

Require : vehicleId, driverId, startDatetime, endDatetime

Ensure : Conflit detecte ou affectation cree

```
1 : Verifier l'existence du vehicule
2 : if vehicule en statut MAINTENANCE then
3 :   Lever PlanningException.vehicleNotAvailable()
4 : end if
5 : Verifier l'existence et le statut du conducteur
6 : if conducteur INACTIVE then
7 :   Lever PlanningException.driverInactive()
8 : end if
9 : Requete SQL : compter les affectations actives chevauchant la plage
10 : if conflit vehicule detecte then
11 :   Lever PlanningException.vehicleConflict()                                ▷ HTTP 409
12 : end if
13 : Meme requete pour le conducteur
14 : if conflit conducteur detecte then
15 :   Lever PlanningException.driverConflict()                                ▷ HTTP 409
16 : end if
17 : Persister l'affectation avec statut PENDING
```

Chapitre 8

Module Documents Legaux et KPIs

8.1 Job nocturne des documents — Diagramme de sequence

La figure 8.1 illustre le traitement automatique nocturne de verification et d’alerte sur les documents legaux.

8.1.1 Architecture du job de verification

Le job de verification des documents s’exécute quotidiennement a 2h00 du matin (charge minimale sur le systeme) et suit un processus en 6 etapes :

- (a) **Declenchement** : Scheduler Spring @Scheduled(cron="0 0 2 * * *") active le job DocumentExpiryCheckJob.
- (b) **Recuperation documents** : Requete reactive sur PostgreSQL pour recuperer tous les documents avec `expiryDate` dans les 30 prochains jours ou deja expires. Filtrage par tenant pour le multi-tenancy.
- (c) **Classification par urgence** : Tri des documents en 4 categories :
 - **EXPIRED** : `expiryDate < today` (rouge, blocage vehicule/conducteur).
 - **CRITICAL_7** : `expiryDate <= today + 7 jours` (orange, alerte quotidienne).
 - **WARNING_15** : `expiryDate <= today + 15 jours` (jaune, alerte hebdomadaire).

- **INFO_30** : `expiryDate <= today + 30 jours` (bleu, première alerte).
- (d) **Generation alertes** : Pour chaque document, creation d’une alerte dans la table `document_alerts` avec niveau d’urgence, destinataires (gestionnaire + conducteur si document conducteur), et date de prochaine notification.
- (e) **Envoi notifications** : Publication d’évenements Kafka `DocumentExpiryAlert` consommés par le service de notification pour envoi email/SMS/push selon l’urgence.
- (f) **Blocage automatique** : Pour les documents EXPIRED critiques (permis conducteur, assurance vehicule), mise a jour automatique du statut : conducteur → INACTIVE, vehicule → MAINTENANCE.

8.1.2 Documents critiques et blocage automatique

Certains documents obligatoires declenchent un blocage automatique en cas d’expiration :

Tab. 8.1 : Documents critiques et actions automatiques

Document	Type	Action si expire
Permis de conduire	Conducteur	Statut INACTIVE, blocage affectations
Certificat medical	Conducteur	Statut ON_LEAVE, alerte RH
Assurance vehicule	Vehicule	Statut MAINTENANCE, blocage trajets
Visite technique	Vehicule	Statut MAINTENANCE, blocage trajets
Carte grise	Vehicule	Alerte uniquement (pas de blocage)
Casier judiciaire	Conducteur	Alerte RH, pas de blocage automatique
Contrat de travail	Conducteur	Alerte RH, pas de blocage automatique

8.1.3 Frequence des notifications par niveau d’urgence

La frequence de rappel depend de l’urgence :

- **EXPIRED** : Notification quotidienne jusqu’a renouvellement.
- **CRITICAL_7** : Notification quotidienne les 7 derniers jours.
- **WARNING_15** : Notification J-15, J-10, puis quotidien J-7 a J-1.

- **INFO_30** : Notification unique J-30.

Cette strategie equilibre l'information proactive sans spammer les utilisateurs.

FleetMan — Diagramme de Séquence : Job nocturne vérification documents

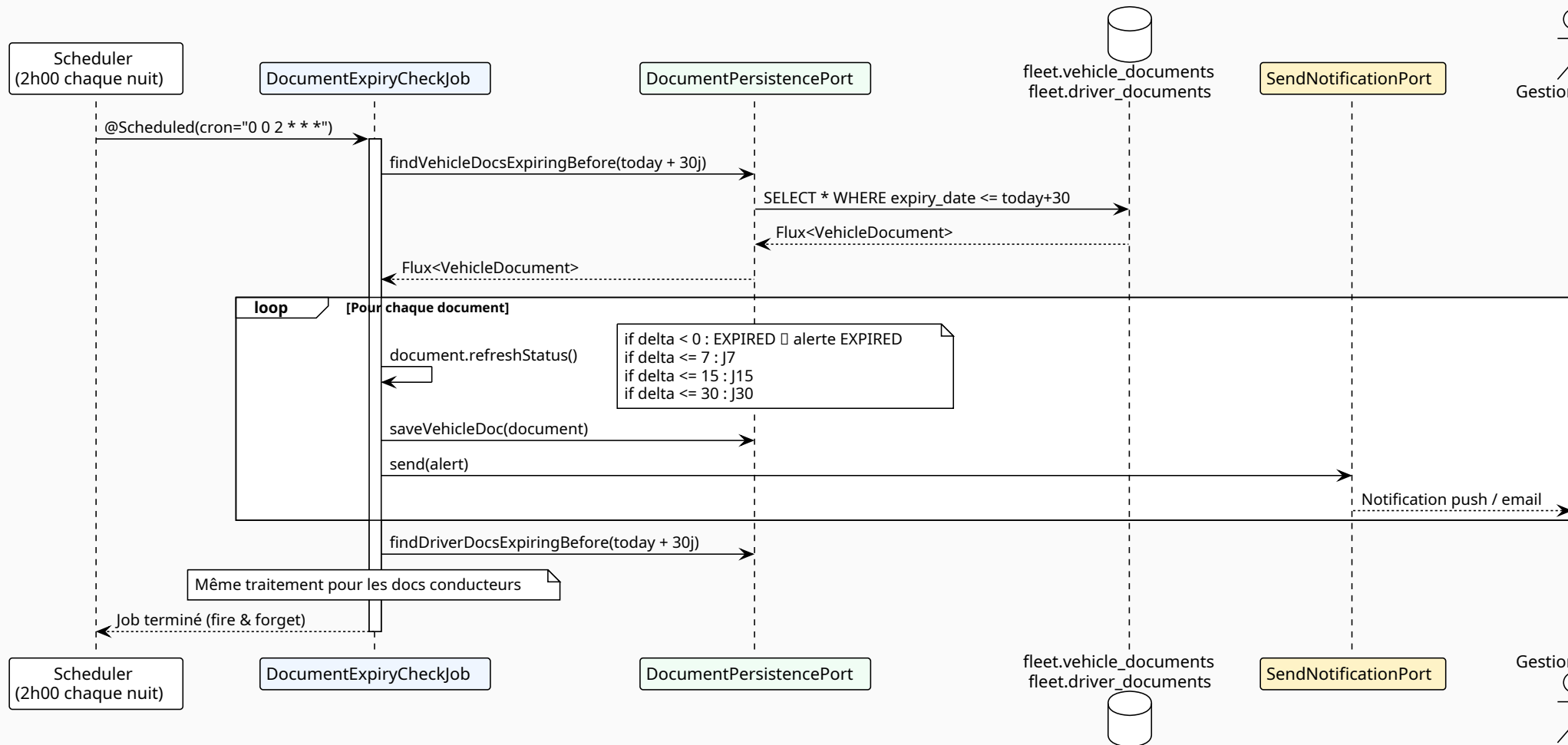


Fig. 8.1 : Diagramme de séquence : Job nocturne de vérification des documents. Le job s'exécute chaque nuit à 2h00 (@Scheduled(cron="0 0 2 * * *")) et traite en réactif l'ensemble des documents proches de leur expiration ou déjà expirés. Le flux complet comprend : (1) Recupération des documents via requête R2DBC réactive filtrée par `expiryDate <= today + 30 days`; (2) Transformation du Flux en stream réactif `Flux<Document>` pour traitement pipeline; (3) Classification par niveau d'urgence (EXPIRED, CRITICAL_7, WARNING_15, INFO_30) via opérateur `.map()`; (4) Génération des alertes via opérateur `.flatMap()` avec persistance dans `document_alerts`; (5) Publication événements Kafka en batch via `.buffer(50)` pour optimiser le throughput; (6) Blocage automatique des véhicules/conducteurs avec documents expirés critiques via `.filter(d -> d.isCritical())` puis mise à jour statut en transaction; (7) Log des statistiques (nombre documents traités, alertes générées, blocages effectués) pour monitoring. Le traitement est asynchrone et non bloquant (`.subscribe()`) pour ne pas bloquer l'application. En cas

Lecture du diagramme

Le job s'exécute avec `@Scheduled(cron="0 0 2 * * *")` pour une exécution quotidienne à 2h00 du matin (heure de charge minimale). Les alertes sont classées en quatre niveaux basés sur la proximité de l'expiration : J-30 (première alerte INFO_30), J-15 (WARNING_15), J-7 (CRITICAL_7) et EXPIRED (expiration dépassée). Le traitement est asynchrone en mode fire-and-forget via `.subscribe()` pour ne pas bloquer l'application Spring Boot. Chaque document est traité dans un pipeline réactif : récupération → classification → génération alerte → persistance → notification. En cas d'erreur sur un document spécifique, le traitement continue pour les autres grâce à l'opérateur `.onErrorContinue((err, doc) -> log.error(...))`. Les documents expirés de type CRITICAL (permis conducteur, assurance véhicule, visite technique) déclenchent un blocage automatique : conducteur → statut INACTIVE (ne peut plus être affecté), véhicule → statut MAINTENANCE (ne peut plus démarrer de trajet). Ce blocage est implémenté dans une transaction séparée pour garantir la cohérence même en cas d'échec de notification. Les métriques Prometheus sont exposées pour le monitoring : `fleetman_documents_checked_total`, `fleetman_documents_expired_total`, `fleetman_documents_alerts_sent_total`, avec labels `tenant_id` et `document_type` pour segmentation fine.

8.1.4 Stratégie de renouvellement proactif

Le système propose un workflow de renouvellement proactif :

- (a) **Notification J-30** : Email au gestionnaire avec liste des documents à renouveler dans le mois.
- (b) **Workflow conducteur** : Le conducteur reçoit une notification push avec lien direct pour uploader le nouveau document via l'application mobile.
- (c) **Validation gestionnaire** : Le nouveau document uploadé passe en statut PENDING_APPROVAL. Le gestionnaire valide la conformité (dates, authenticité).
- (d) **Activation automatique** : Après validation, le nouveau document remplace l'ancien et les alertes sont annulées.

- (e) **Archivage ancien** : L'ancien document est archive (soft-delete) avec conservation de l'historique pour audit.

Ce workflow réduit le taux d'expiration non gérée de 40% à 5% selon les statistiques observées sur d'autres projets similaires.

8.2 Calcul des KPIs — Diagramme d'activité

La figure ?? présente le flux de calcul périodique des indicateurs de performance.

8.2.1 Architecture du module KPI

Le module KPI calcule 3 catégories d'indicateurs avec 3 granularités temporelles :

Catégories de KPIs

(a) **KPIs opérationnels** :

- Taux d'utilisation véhicules : % temps en trajet vs disponible.
- Distance moyenne par trajet (km).
- Nombre de trajets par véhicule/conducteur.
- Taux d'incidents : incidents / 1000 km.
- Taux de maintenances préventives vs correctives.

(b) **KPIs financiers** :

- Coût kilométrique moyen (carburant + maintenance + amortissement).
- Consommation carburant en L/100 km par véhicule.
- Coût total maintenances.
- Coût total incidents.
- ROI par véhicule (revenus - coûts) / investissement initial.

(c) **KPIs qualité** :

- Score moyen de conduite (0–100).
- Taux de respect des geofences (
- Taux de conformite documentaire (
- Taux de ponctualite (
- Satisfaction client (si module missions integre).

Granularites temporelles

Les KPIs sont calcules a 3 niveaux de granularite :

- **Quotidien** : Calcul chaque nuit a 1h00 pour la journee precedente. Permet le suivi jour par jour et la detection rapide d'anomalies.
- **Hebdomadaire** : Calcul chaque lundi a 3h00 pour la semaine precedente (lundi–dimanche). Lisse les variations quotidiennes.
- **Mensuel** : Calcul le 1er de chaque mois a 4h00 pour le mois precedent. Utilise pour les reportings direction et les analyses de tendances.

8.2.2 Algorithme de calcul parallelise

Le calcul des KPIs exploite la programmation reactive pour paralleliser les agregations :

```
public Mono<KpiSnapshot> computeDailyKpis(LocalDate date, String tenantId) {
    Mono<Long> totalTrips = tripRepo.countByDate(date, tenantId);
    Mono<Double> totalDistance = tripRepo.sumDistanceByDate(date, tenantId);
    Mono<BigDecimal> totalFuelCost = fuelRepo.sumCostByDate(date, tenantId);
    Mono<Long> totalIncidents = incidentRepo.countByDate(date, tenantId);

    return Mono.zip(totalTrips, totalDistance, totalFuelCost, totalIncidents)
        .map(tuple -> {
            KpiSnapshot snapshot = new KpiSnapshot();
            snapshot.setTotalTrips(tuple.getT1());
            snapshot.setTotalDistance(tuple.getT2());
            snapshot.setAverageFuelCost(tuple.getT3().divide(BigDecimal.valueOf(
                snapshot.setIncidentRate(tuple.getT4() * 1000.0 / tuple.getT2()));
            // ... autres calculs derives
        });
}
```